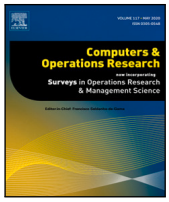
Computers & Operations Research 137 (2022) 105556

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor



Wave order picking under the mixed-shelves storage strategy: A solution method and advantages

Seyyed Amir Babak Rasmi [a,*], Yuan Wang [b], Hadi Charkhgard [c]

[a] School of Electrical Engineering, Computing and Mathematical Sciences, Curtin University, Perth, Australia
[b] School of Business, Singapore University of Social Sciences, Singapore 599494, Singapore
[c] Department of Industrial & Management Systems Engineering, University of South Florida, Tampa, FL, 33620, USA

ARTICLE INFO

Keywords:
Order picking
Order batching
Multiple pickers' routing
Bi-objective MILP
Mixed-shelves storage strategy
Makespan minimization

ABSTRACT

Order picking (OP) is a time and cost consuming operation in many warehouses. The optimization of order picking planning is crucial to speed up customers' delivery process and reduce warehouse expenses. A commonly adopted strategy is called mixed-shelves storage strategy (MSSS) that is highly recommended for e-commerce warehouses. Applying this, the units of the same stock keeping unit (SKU) are scattered to various storage locations which provides more improvement room for OP. However, consolidated methods have not been sufficiently discussed when OP is under the MSSS. In this paper, we study the problem of wave picking systems to jointly address order batching, batch assignment, and picker routing (BAR) in a MSSS-based warehouse. We propose the decomposition of OP into BAR subproblems (DOPBAR) for solving such a wave OP problem. We examine our approach on a large set of instances, validate the effectiveness of our method, and point out the advantages of the MSSS in OP. Moreover, we analyze the trade-offs between two conflicting objectives that practitioners aim to optimize: customer service level (makespan) and workforce level (labor cost). Finally, a few insightful managerial recommendations are proposed that support the decision making in choosing a storage location assignment strategy.

1. Introduction and literature review

1.1. Introduction

In spite of the existence of many services and products that are digitally provided in nowadays' marketplace environment, there exist a huge number of product types that must be physically delivered to the consumers. High expectations of the buyers of these goods trigger the sellers to pay particular attention to warehousing various SKUs. Therefore, warehouse operations management – including receiving, storage, order picking (OP), and shipping (Gu et al., 2007) – remains an attractive topic for the researchers and practitioners (Boysen et al., 2019). OP is the "*process of retrieving products from storage (or buffer areas) in response to a specific customer request*" (De Koster et al., 2007), and has been one of the key topics in automated, semi-automated, and manual warehousing systems. The expenses of this process occupy approximately 55% of the total cost in warehouse operations (De Koster et al., 2007). Thus, OP is crucial in both customer service level and the total cost in warehousing, and multiple circumstances affect this process.

There are various highly correlated sub-planning problems and strategies in OP planning. For instance, storage location assignment (SLA) is one of them that can considerably influence OP planning. Warehouses apply different SLA strategies to store various SKUs. In terms of scattering the units of a SKU in a warehouse, there mainly exist two SLA strategies: *dedicated* storage strategy (DSS) and *mixed-shelves* storage strategy (MSSS) (see De Koster et al. (2007), Chen et al. (2010), and Roodbergen (2012) for more discussion on other SLA strategies). Based on the DSS, all units of the same SKU are stored in one storage location or in multiple storage locations next to each other. When the MSSS applies, the units of the same SKU are scattered all around the warehouse. This SLA strategy is also known as *shared/scattered* storage strategy and has several pros and cons (Boysen et al., 2019). Applying each of these strategies results in different outcomes for an OP process (Weidinger and Boysen, 2018; Petersen et al., 2017). To the best of our knowledge, Weidinger and Boysen (2018) and Jiang et al. (2020) are the only studies for finding an optimal SLA under the MSSS. Weidinger and Boysen (2018) develop a mathematical model for finding the optimal SLA for a MSSS-based warehouse which minimizes the maximum travel time (TT) that may be required for collecting an order. Jiang et al. (2020) include correlation between a pair of SKUs

* Corresponding author.
E-mail address: babak.rasmi@curtin.edu.au (S.A.B. Rasmi).

https://doi.org/10.1016/j.cor.2021.105556
Received 29 January 2021; Received in revised form 23 August 2021; Accepted 7 September 2021
Available online 11 September 2021

and minimize the weighted-sum TTs of all product pairs to reduce the total TT.

By implementing the MSSS, finding a shorter tour for a picker is highly probable as newly received units of one SKU are scattered to any suitable storage location. However, the MSSS adds complexity to a picker's routing problem as for each SKU multiple storage locations exist as the alternatives. Let all bins that contain the same SKU be represented as a single picking node due to their adjacency under the DSS. Then, finding the shortest route can be formulated as a traditional traveling salesman problem (TSP). Ratliff and Rosenthal (1983) present a method for solving such a TSP in a rectangular single-block warehouse with a polynomial computational time which is linear in the number of aisles. On the other hand, in the presence of the MSSS, this TSP changes to a different problem and becomes more complicated. Considering the fact of several scattered bins for each SKU, Weidinger (2018) proves that this problem is strongly NP-hard, and methods for minimizing one picker's tour length are developed (Daniels et al., 1998; Weidinger, 2018; Weidinger et al., 2019). However, OP planning with multiple pickers under the MSSS is still challenging as order batching, batch assignment, and picker coordination are vital. We illustrate on potential efficiency improvements derived from the MSSS to obtain shorter routes in Fig. 1.

Figs. 1(a) and 1(b) show two single-block warehouses including three vertical picking aisles and two horizontal cross aisles. The number of units of a SKU available for each SKU is equal in both warehouses; then, an equal inventory level applies. Fig. 1(a) is under the DSS, and Fig. 1(b) is based on the MSSS. Let the number in each gray box denote the SKU number, and each bin contains three units of that SKU. An order is given with two units of SKU 5, one unit of SKU 6, and one unit of SKU 10 (the numbers in the bins that correspond to these SKUs are bolded). We assume that a picker starts and ends his/her tour at the depot, and every two bins in the same picking aisle in front of each other are accessible from a single picking node (e.g., black nodes in this figure). Then, for retrieving the requested units/items of SKUs in Fig. 1(a) (i.e., the DSS based warehouse), a picker has to move a larger distance than his/her tour length in Fig. 1(b). Note that for an order such as twelve units of SKU 10, the outcome will be different in Fig. 1(b); a picker will be supposed to visit four bins that contain SKU 10 and are not close to each other. In Fig. 1(c), we show a graph that represents the connections (denoted by dashed lines) between the picking nodes, the depot (denoted by Ⓓ), and the vertices in the cross aisles (denoted by ×). Let g, u, and m denote the TT between various vertex types in the graph representation of the warehouse given in Figs. 1(a, b). Later on, we use these parameters to establish a warehouse setting.

Despite more OP planning challenges under the MSSS, its potentials for improving OP efficiency encourage many warehouses to exploit the MSSS rather than the DSS. These warehouses are mainly operated in business-to-consumer (B2C) markets and receive orders with particular behaviors. Their customers usually request units of one/a few SKUs in small quantities (e.g., 1 or 2). In recent years, due to this demand structure in B2C markets, the MSSS has become more prevalent among online retailers (e.g., Amazon). They have found the MSSS as an approach to collect many orders from a warehouse in a short period of time and ship the wave out (Weidinger et al., 2019). In order to take full advantage of possible improvements that inherently exist in the MSSS, we have to extend MSSS analysis beyond the existing studies which only aim to find the shortest tour of one picker (Weidinger et al., 2019; Weidinger, 2018). In particular, order batching is inevitably required as the number of orders is always significantly larger than the number of available pickers. In that situation, batch assignment and picker coordination become more crucial because the units of a SKU are scattered to various storage locations with limited inventory levels. When a warehouse operates under the MSSS, decisions about BAR are mutually connected with picker coordination, and hence, they influence the total retrieval time, and consequently the makespan (i.e., the maximum RT for any of pickers). Note that retrieval time (RT) includes both TT and picking times at storage locations. See Appendix A for an illustrative example on order batching and picker routing.

As discussed above, integrating BAR sub-planning problems for finding the optimal solution in an OP process is crucial yet very challenging. This joint decision-making on three highly correlated sub-planning problems for OP is addressed under the DSS in a number of studies (e.g., see Van Gils et al. (2019a), Ardjmand et al. (2018), Valle et al. (2017), Menéndez et al. (2017), Henn (2015), Matusiak et al. (2014), and Kulak et al. (2012)). However, this joint problem is not comprehensively explored in a MSSS-based warehouse for achieving the advantages of the MSSS. The only study that has approached this problem is Yang et al. (2020). That paper proposes a method for batching orders and routing one picker in a MSSS-based warehouse to minimize the total travel distance. However, it does not consider the existence of multiple pickers and assumes there is an infinite inventory available for the units of a SKU in its corresponding storage locations. Hence, picker coordination is not required, and infinite inventory level reduces the complexity of OP by making that unrealistic for a wide range of practical cases. Moreover, in the literature of the MSSS, the impact of the MSSS on the trade-offs between the number of pickers and the makespan is not evaluated and is still a gap. Boysen et al. (2019) also state that the MSSS still requires more attention.

In this paper, we assume that an existing MSSS-based SLA is given as it is common in mixed-shelves warehouses that deciding about SLA has always been made before orders arriving. We seek a method for optimizing a wave OP plan to minimize the makespan and the workforce level as two objective functions in our bi-objective optimization model. These two are conflicting with each other and wished for achieving a high service level to the customers with a smaller workforce size. In fact, we can obtain a shorter makespan with a larger workforce size. In this paper, we do not aim to depict the Pareto frontier of our bi-objective optimization model. However, we analyze how the makespan value changes in workforce size to provide insights. We point out the main contributions in this paper as follows:

- We formulate a bi-objective mixed-integer linear program (MILP) named [OPMSSS] that integrates BAR under the MSSS.
- We develop solution scheme to solve [OPMSSS] by using a clustering-based method to batch orders and assign them to pickers.
- We examine our proposed method on a large number of SLA instances and orders randomly generated.
- Based on the investigated SLA instances, we evaluate the performance of a number of SLA strategies and their variants to retrieve items requested by a range of selected order profiles.
- We analyze the outcomes of numerical experiments and present a few managerial/operational insights. We apply our method to examine the pros and cons of the MSSS versus the DSS and observe the trade-offs between the customer service level (i.e., the makespan) and the workforce level (i.e., the labor cost). These analyses are crucial in OP systems (Daniels et al., 1998).

The remaining sections in this paper are organized as follows. In Section 1.2, we review a number of existing studies in OP. In Section 2, we describe the wave OP planning problem we focus on in this paper and propose a mathematical formulation. We solve that model using a method explained in Section 3. We show our method for generating random problem instances in Section 4, and present our numerical results in Section 5. Then, we analyze those results to obtain insightful recommendations in Section 6 that is followed by conclusions in Section 7. We also provide appendices at the end of this paper.

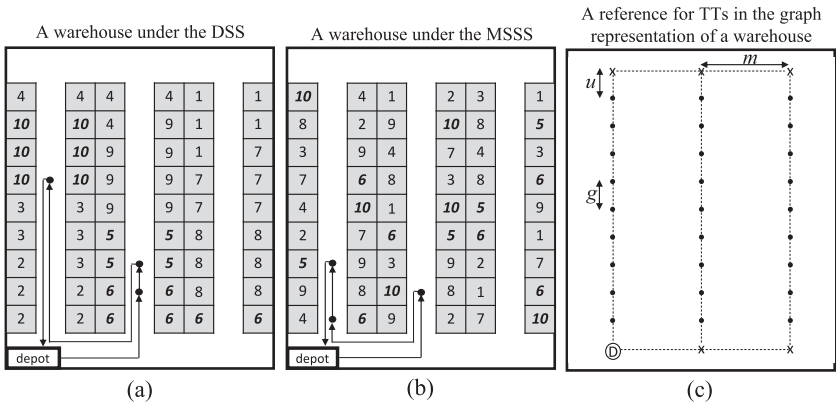


Fig. 1. Illustrative example for comparing OP process for an order that requests units of SKUs 5, 6, and 10 under (a) the DSS and (b) the MSSS; (c) is given as a reference for the TTs between the vertices in the graph representation.

1.2. Literature review

As OP is a crucial activity in warehouses, there is a wide literature on that. Studies in OP mostly focus on two solution approaches: optimization and mathematical programming based (OMPB) studies – in which we are interested – and simulation based (SB) studies. Some areas have been addressed by recent SB studies: for example, safety constraints, picker blocking (Van Gils et al., 2019b), real-time order arrival (Lu et al., 2016), warehouse layout, total waiting time (Yener and Yazgan, 2019), the effectiveness of bucket brigades OP in a multi-aisle warehouse (Quader and Castillo-Villar, 2018), the impacts of storage strategies on picker blocking (Pan and Wu, 2012), and the significance of integrating OP sub-planning problems (Van Gils et al., 2018a, 2019b). Due to the wide literature on OMPB studies, we do not discuss all basic issues and definitions as in-depth reviews are available (see Boysen et al. (2019), Van Gils et al. (2018b), De Koster et al. (2007), and Gu et al. (2007)). Instead, we mostly focus on the areas relevant to our study.

A large proportion of OP systems are picker-to-part (De Koster et al., 2007). In these systems, the pickers move through the aisles with respect to the constraints exposed by the warehouse infrastructure, visit some storage locations, and collect the requested items. Van Gils et al. (2018b) acknowledge that routing pickers, batching orders, zoning, and SLA have attracted the researchers' and practitioners' attention more than other planning problems in OP.

As briefly discussed in Section 1.1, it is accepted that some SLA strategies let pickers find shorter routes to retrieve orders. Although there are OMPB implementations for SLA to decrease pickers' TTs/RTs (e.g., see Silva et al. (2020)), a widely-applied approach for the DSS is the class-based storage strategy. In this regard, based on the historical demand, all SKUs are grouped to usually three classes (so-called A, B, and C) using Pareto's method. Then, each SKU is assigned to one or a number of storage locations based on its class. Petersen and Aase (2004) and Roodbergen (2012) show that applying the within-aisle or across-aisle class-based storage strategy under the DSS results in significant savings while retrieving orders. In terms of finding an optimal SLA, Larco et al. (2017) provide SLA solutions that consider both minimizing the RT of an order and minimizing pickers' discomfort, simultaneously. Weidinger and Boysen (2018) and Jiang et al. (2020) are the only studies that propose an optimized SLA under the MSSS for minimizing pickers' traveling activities. In terms of data mining approaches, Pang and Chan (2017) present a method for finding a SLA considering historical data. Yener and Yazgan (2019) and Reyes et al. (2019) scrutiny the literature of SLA strategies and admit that there is a limited literature devoted to the MSSS.

BAR are fundamental well-known sub-planning problems in OP. There are many contributions that solely focus on one of them. Due to their wide literature, we mention a few pioneer studies; for instance, in batching: Chen and Wu (2005) and Žulj et al. (2018); and in picker routing: Ratliff and Rosenthal (1983), Roodbergen and Koster (2001), and Weidinger (2018).

The solutions of the mentioned sub-planning problems are supportive for the decision makers in the level of tactical and operational decisions. However, as far as all planning problems are not integrated into one, the obtained solutions are locally optimal. Therefore, more significant improvements are possible using integration (Van Gils et al., 2018a). On the other hand, many real-life constraints exist that make OP more challenging; approaching these constraints is a crucial requirement. In most warehouses, cross aisles are sufficiently wide such that no congestion occurs within them. However, narrow picking aisles restrict pickers' moves. When multiple pickers work at the same time, picker blocking may become a critical real-life constraint that is discussed by Hong et al. (2012a) and Van Gils et al. (2019b) in detail.

Table 1 briefly presents the recent studies that integrate OP-related sub-planning problems. In many cases, addressing the same combination of planning problems does not result in exactly the same problem since each of them might have been defined based on different assumptions. Moreover, some combinations are not reasonable for consideration. For instance, warehouse layout is a strategic long-term decision and cannot be combined by order batching that is a daily operational decision.

Table 1 shows that recent publications mostly address BAR problems jointly. These three sub-planning problems (batching, assignment, and routing) are crucial for all picker-to-part OP systems. As seen in this table, minimization of total TT and tardiness are the most common objective functions. Moreover, due to the complexity of OP, non-exact approaches are widely applied: e.g., Tabu search (TS), simulated annealing (SA), genetic and memetic algorithms (GA and MA), variable neighborhood search (VNS), particle swarm optimization (PSO), variable neighborhood descent (VND), ant colony optimization (ACO), iterated local search (ILS), and hybrid methods. Besides the mentioned points in Table 1, there exist specific real-world considerations that are addressed in some studies. For example, they include picker blocking, heterogeneous picking vehicles, and 3-dimensional warehouses with multiple levels that are marked for some of the papers referred to in this table.

As seen, there exist a large number of studies in OP optimization that focus on (meta)heuristics because real-world warehouses are facing a huge number of products/orders which decreases the effectiveness of exact optimization methods. On the other hand, these non-exact methods mainly mentioned in Table 1 have disadvantages; e.g., a right set of algorithm-related parameters needs to be investigated for capturing a desirable level of solution quality. These settings are also subject to the warehouse's layout/constraints and orders' profile. Moreover, the complexity of OP problems makes their solutions unknown in some

Table 1
Recent OMPB studies that integrate more than one OP sub-planning problem.

Study	MSSS	Order batching	Picker routing	Other considerations	Minimization objective function(s)	Method(s)
Parikh and Meller (2008)		✓		picker blocking, work imbalance, zoning	total cost	examining the results of optimization methods on a cost function
Kulak et al. (2012)		✓	✓	–	total TT	TS integrated with a clustering algorithm
Hong et al. (2012a)		✓	✓	picker blocking	total RT (loading/unloading, walk, delay)	SA
Hong et al. (2012b)		✓	✓[a]	batch sequencing	total TT	heuristics
Azadnia et al. (2013)		✓	✓[a]	–	total tardiness	weighted association mining and GA
Matusiak et al. (2014)		✓	✓	–	total TT	a hybrid of SA and a routing method
Henn (2015)		✓	✓	batch sequencing	total tardiness	VNS
Cheng et al. (2015)		✓	✓	–	total TT	a hybrid of PSO and ACO
Cortés et al. (2017)			✓	multiple pickers who use heterogenous equipments	total RT (setup, walk, picking)	TS
Scholz et al. (2017)		✓	✓[a]	batch sequencing	total tardiness	VND
Valle et al. (2017)		✓	✓	optimal solution in a 3D layout	total TT	adding valid inequalities for tightening the feasible region
Dijkstra and Roodbergen (2017)			✓	optimizing SLA based on certain routing methods	exact average route length	finding the exact average route length based on the probability of at least one pick in an aisle
Li et al. (2017)		✓	✓	3D layout	total TT	similarity coefficient techniques with ACO
Menéndez et al. (2017)		✓	✓	batch sequencing	total tardiness	VNS
Ardjmand et al. (2018)		✓	✓	batch sequencing	makespan	SA and ACO
Chen et al. (2018)		✓		orders in a 3D warehouse arrive in real time and can be split	pickers' idle time	a heuristic named Green Area
Miguel et al. (2019)		✓	✓	–	total cost including penalty for delays	MA
Cano et al. (2019)		✓	✓	–	total TT/tardiness	no method
Weidinger et al. (2019)	✓	✓	✓	single picker	total TT	nearest neighbor and pool-based construction heuristics
Van Gils et al. (2019a)		✓	✓	–	total TT	ILS
Silva et al. (2020)			✓	optimizing SLA for a certain set of orders	total TT	general VNS
Yang et al. (2020)	✓	✓	✓	no picker coordination, infinite inventory for the units of a SKU at a storage location	total TT	combination of multiple heuristic methods
Briant et al. (2020)		✓	✓	applied on industrial cases	total TT	column generation
Yousefi Nejad Attari et al. (2020)		✓	✓	multiple planning periods and considering the solution robustness	total cost including the fixed cost of using warehouse vehicles	PSO and GA
Haouassi et al. (2021)		✓	✓	the impact of splitting orders is considered	total TT	a 2-phase metaheuristic with clustering
This paper	✓	✓	✓	picker coordination, batch assignment	makespan based on a given number of pickers	DOPBAR

[a]They select a route among a number of options of predetermined tours.

cases, and hence, the effectiveness of applied (meta)heuristics becomes unclear to the researchers and practitioners.

Other than the aspects mentioned for OP planning problems, there exists another side for these problems that explains how B2C warehouses treat their arriving orders. In some businesses, a warehouse manager needs to consider dynamic order arrivals and make a real-time batch/assign decision for them (refer to Gil-Borrás et al. (2020) and Zhang et al. (2017) as the most recent studies in this area). This setting is highly applicable when the warehouse can forecast the orders' arrival times in advance and there is a due date for each order. Therefore, some orders should be prioritized which may influence batching/assignment decisions. On the other hand, many warehouses quote a common delivery time to a large number of orders in a certain time window (i.e., a wave of orders). Hence, these warehouses consider all orders received in a certain time window and look for an OP plan to retrieve the orders from their warehouse facilities in the shortest possible time.

2. Problem description and formulation

We integrate BAR problems in a picker-to-part OP system when the MSSS applies as the SLA strategy of a warehouse. That warehouse includes a number of aisles surrounded by racks established on one or multiple blocks (our proposed method will work for a warehouse with a general layout as long as its TT matrix can be provided; however, we will test that only for single-block warehouse instances to focus on our goals). If a warehouse with relatively narrow aisles only has one level, at each picking node, there are two storage locations or bins:

one bin in left and one in right. Then, a picker can access to both bins from each picking node. Each bin contains a number of units available only of one SKU, and there may exist more than one bin containing a particular SKU.

Let an e-commerce based company be taken into account. Then, we are interested in the wave OP process, and that company everyday (or every certain few hours/days) receives a number of orders which should be fulfilled by a common deadline. In most cases, the company is committed to provide a delivery service based on "next-day delivery" concept (e.g., "Order now, deliver tomorrow!" and the like). The aforementioned deadline is actually originated from the service type the company offers to its customers. After receiving (a sufficient number of) orders, the company's warehouse starts to collect the items. The warehouse is supposed to coordinate its pickers for retrieving the requested units of SKUs – that is assumed as a wave of orders – in the shortest possible makespan. This approach speeds up the shipment of that wave and releases the pickers earlier for either picking the next wave or continuing their jobs in other sections.

In many cases, the number of orders in a wave OP process is significantly larger than the number of pickers. Therefore, every number of orders are batched and assigned to one picker for retrieving in one tour that starts and ends at the depot. Each picker may need to collect multiple batches and retrieves the orders assigned to his/her driving a trolley through the aisles. Moreover, the approach we are interested in, *sort-while-pick*, does not require to be followed up by a long sorting procedure after OP and before shipment. For collecting the items in one batch in the shortest possible RT, the corresponding picker must move through the shortest route and be coordinated with other pickers to prevent stock-out in the bins.

Let some routes be chosen for the pickers. These determined routes answer to the following questions:

- Which bins are supposed to be visited?
- How many units of the corresponding SKU must be collected?
- Who has ordered the items?

Then, each picker starts at the depot, walks through the aisles/picking nodes, visits several storage locations, and picks some units of various SKUs. We assume that the total RT for a picker consists of two elements: 1- the total TT which corresponds to the total distance of the picker's tour, and 2- the time required for picking items which corresponds to the number of the storage locations visited. In our assumption, picking time at one storage location is regardless of the number of units that must be retrieved.

In order to take both customer service level and workforce level into account, we minimize two objective functions at the same time: the makespan and the number of pickers that inherently conflict with each other (i.e., more pickers, a faster OP). Makespan is an important criterion due to one direct and one indirect result. Minimizing the makespan directly guarantees to capture the minimum time that a wave OP process can be completed deploying a certain level of workforce. In terms of the indirect result, minimizing the makespan provides a more balanced workload for all pickers whereas this result is not captured by minimizing the total time spent by all pickers on OP.

We also consider some other assumptions in our problem as follows:

- *No correlation among the orders:* We assume that there are no significant correlations among the demanded SKUs and the requested number of units. This assumption is applied to the SKUs within an order and between a pair of orders. A significant correlation may make some SLA strategies irrational.
- *No stock-out:* The demanded number of units for each SKU does not exceed the total number of the units available in the warehouse. Thus, stock-out does not occur.
- *No preference among the units of a SKU:* In some industries, some products must be retrieved earlier due to their expiry dates, weights, etc. However, we assume that there is no difference between any two units of one SKU.
- *Available information on the storage locations:* The warehouse manager has information on the SKUs' locations and available number of units in each bin.
- *No splitting orders:* We use sort-while-pick strategy in which splitting orders to more than one batch is not possible.
- *All orders with the same deadline:* There is no preference to retrieve particular orders earlier. In other words, when all orders are completed, they are shipped out together.
- *Zero TT between two storage locations in one aisle and in front of each other:* If two storage locations are in the same aisle and can be accessed via one picking node, the TT between them is assumed to be zero.

Regarding "All orders with the same deadline", note that considering a common deadline is not applicable for the warehouses dealing with a continuous flow of orders that must be fulfilled within different deadlines. However, this assumption is applicable for a wide range of problems in the e-commerce where most of the orders made during a certain period are to be met by a guaranteed quoted delivery time. Moreover, another point regarding our assumption is that many warehouses have to satisfy a wave of orders before starting the next wave collection. However, a warehouse may face multiple waves with different deadlines to be retrieved simultaneously. In this case, the stated assumptions and our model need changes to address this setting.

When it comes to the formulation of an OP planning problem, it can be interpreted as a non-preemptive parallel machine/job scheduling problem. Hereby, machines represent pickers, and each job is shown by multiple picks that should be performed. Unlu and Mason (2010) categorize mixed-integer programs for the non-preemptive parallel machine/job scheduling problems into four basic models. Although we require to capture several factors and constraints for the problem described in the last section, the basic model design proposed by Unlu and Mason (2010) as "network model for scheduling parallel machines" addresses the main pillars of our model, i.e., both routing and assigning jobs/batches to multiple machines/pickers at the same time. We apply the notation explained in Table 2 to formulate our problem by the expressions given in (1) and (2) as two objective functions subject to constraints (3)–(22). In order to clarify our notation, note that each storage location is denoted by a unique v_i^k (the kth storage location that contains SKU i). Moreover, let $v_{0^s}^0$ and $v_{0^e}^0$ denote the depot for the moves of a picker for departing from and ending at the depot, respectively.

Weidinger et al. (2019), Van Gils et al. (2019a), Valle et al. (2017), and Hong et al. (2012a) present MILPs that integrate a number of OP sub-planning problems under certain considerations. In this section, we exploit them and present a new bi-objective MILP in line with the described problem. Our formulation given in [OPMSSS], integrates BAR considering the case that there may exist more than one storage location for a certain SKU. Note that we assume at least one unit of each SKU is requested (i.e., $\sum_{o\in O} n_o^i > 0,\ \forall i \in I$). Otherwise, one can reduce the size of the MILP by eliminating the storage locations that correspond to the SKUs undemanded. Moreover, trolley capacity must be greater than or equal to $\sum_{i\in I} n_o^i,\ \forall o \in O$, to make the OP process possible (note that we cannot split an order to more than one batch). Our formulation is as follows:

[**OPMSSS**] :

$$\min\ T_{max} \tag{1}$$

$$\min\ \sum_{r\in R} A_r \tag{2}$$

$$\text{s.t.}\ \sum_{b\in B} Y_b^o = 1,\ \forall o \in O, \tag{3}$$

$$\sum_{o\in O}\sum_{i\in I} n_o^i Y_b^o \le Cap_{unit} W_b,\ \forall b \in B, \tag{4}$$

$$\sum_{r\in R} U_b^r = W_b,\ \forall b \in B, \tag{5}$$

Table 2
The description of the sets, indices, parameters, and decision variables used in [OPMSSS].

Sets, indices, and parameters	
I	Set of all SKUs in the inventory (indices: i, j, and ι)
I_i	Set of the indices that distinguish the storage locations which contain at least one unit of SKU i (indices: k, p, and s).
v_i^k	The kth storage location of SKU i or the depot (s.t. $i \in I \cup \{0^s, 0^e\}$ and $k \in I_i$ where $I_{0^s} := \{0\}$ and $I_{0^e} := \{0\}$)
q_i^k	The number of units of SKU i available in storage location v_i^k
O	Set of all orders (indices: o and o')
S_o	Set of the SKUs requested by order o
S^i	Set of orders by which SKU i is requested
B	Set of batches' indices starting from 1 to $\lvert B\rvert$ (indices: b and b'). Note that $\lvert B\rvert \le \lvert O\rvert$.
R	Set of pickers' indices starting from 1 to $\lvert R\rvert$ (indices: r and r')
$\tau_{v_i^k v_j^p}$	TT from bin v_i^k to bin v_j^p for a picker
$T_{pick}(i)$	A function that gives a picking time at a bin which contains SKU i. We assume that $T_{pick}(i) := \tau_{pick} > 0$, $\forall i \in I$, and $T_{pick}(i) := 0$ if $i \notin I$.
Cap_{unit}	Capacity of one trolley used for collecting one batch in terms of the number of units of SKUs
n_o^i	Number of units of SKU i requested by order o
$\hat{M}$	A sufficiently large positive number
Decision variables	
$X^b_{v_i^k v_j^p}$	1 if v_j^p is immediately visited after v_i^k in batch b. Otherwise, 0 (s.t. $i \in I \cup \{0^s\}$, $k \in I_i$, $j \in I \cup \{0^e\}$, $p \in I_j$, and $b \in B$).
Y_b^o	1 if order o is assigned to batch b. Otherwise, 0 (s.t. $o \in O$ and $b \in B$).
W_b	1 if batch b is active (not empty). Otherwise, 0 (s.t. $b \in B$).
U_b^r	1 if batch b is assigned to picker r. Otherwise, 0 (s.t. $b \in B$ and $r \in R$).
A_r	1 if picker r is active. Otherwise, 0 (s.t. $r \in R$).
Q_{ib}^{ko}	Number of SKU i's units picked at v_i^k for meeting order o in batch b (s.t. $i \in I$, $k \in I_i$, $b \in B$, and $o \in O$).
T^b	Total RT for batch b (s.t. $b \in B$).
$\bar{T}^r$	Total RT for picker r (s.t. $r \in R$).
$\hat{T}^{br}$	Total RT for picker r to collect batch b (s.t. $b \in B$ and $r \in R$). Indeed, this variable is equal to $T^b U_b^r$.
T_{max}	Makespan
$F^b_{v_i^k}$	Amount of time between the moment a picker for collecting batch b left the depot and arrives v_i^k to perform a pick (s.t. $i \in I \cup \{0^s, 0^e\}$, $k \in I_i$, and $b \in B$).

$$\sum_{b\in B} U_b^r \le |B| A_r, \ \forall r \in R, \tag{6}$$

$$\sum_{j\in I}\sum_{p\in I_j} X^b_{v_{0^s}^0 v_j^p} = W_b, \ \forall b \in B, \tag{7}$$

$$\sum_{i\in I}\sum_{k\in I_i} X^b_{v_i^k v_{0^e}^0} = W_b, \ \forall b \in B, \tag{8}$$

$$\sum_{j\in I}\sum_{p\in I_j\setminus\{k:j=i\}} X^b_{v_i^k v_j^p} \le W_b, \ \forall b \in B, i \in I, k \in I_i, \tag{9}$$

$$\sum_{i\in I\cup\{0^s\}}\sum_{k\in I_i\setminus\{p:i=j\}} X^b_{v_i^k v_j^p} = \sum_{t\in I\cup\{0^e\}}\sum_{s\in I_t\setminus\{p:t=j\}} X^b_{v_j^p v_t^s}, \tag{10}$$
$$\forall b \in B, j \in I, p \in I_j,$$

$$\sum_{k\in I_i} Q_{ib}^{ko} = n_o^i Y_b^o, \ \forall b \in B, o \in O, i \in I, \text{ if } n_o^i \neq 0, \tag{11}$$

$$\sum_{o\in S^i}\sum_{b\in B} Q_{ib}^{ko} \le q_i^k, \ \forall i \in I, k \in I_i, \tag{12}$$

$$\Big(\sum_{o\in O} n_o^i\Big)\Big(\sum_{j\in I\cup\{0^e\}}\sum_{p\in I_j\setminus\{k:j=i\}} X^b_{v_i^k v_j^p}\Big) \ge \sum_{o\in S^i} Q_{ib}^{ko}, \tag{13}$$
$$\forall b \in B, i \in I, k \in I_i,$$

$$\sum_{j\in I\cup\{0^e\}}\sum_{p\in I_j\setminus\{k:j=i\}} X^b_{v_i^k v_j^p} \le \sum_{o\in S^i} Q_{ib}^{ko}, \ \forall b \in B, i \in I, k \in I_i, \tag{14}$$

$$F^b_{v_i^k} + \tau_{v_i^k v_j^p} + T_{pick}(i) - F^b_{v_j^p} \le \hat{M}(1 - X^b_{v_i^k v_j^p}), \tag{15}$$
$$\forall b \in B, i \in I \cup \{0^s\}, j \in I \cup \{0^e\}, k \in I_i, p \in I_j,$$

$$F^b_{v_{0^e}^0} \le T^b, \ \forall b \in B, \tag{16}$$

$$\sum_{b\in B} T^b U_b^r = \bar{T}^r, \ \forall r \in R, \tag{17}$$

$$\bar{T}^r \le T_{max}, \ \forall r \in R, \tag{18}$$

$$X^b_{v_i^k v_j^p} \in \{0,1\}, \ \forall b \in B, i \in I \cup \{0^s\}, \tag{19}$$
$$j \in I \cup \{0^e\}, k \in I_i, p \in I_j,$$

$$Y_b^o, U_b^r, W_b, A_r \in \{0,1\}, \ \forall b \in B, o \in O, r \in R, \tag{20}$$

$$Q_{ib}^{ko} \in \mathbb{Z}^+ \cup \{0\}, \ \forall b \in B, o \in O, i \in I, k \in I_i, \text{ if } n_o^i \neq 0, \tag{21}$$

$$F^b_{v_i^k}, T^b, \bar{T}^r, T_{max} \in \mathbb{R}^+ \cup \{0\}, \tag{22}$$
$$\forall b \in B, i \in I \cup \{0^s, 0^e\}, k \in I_i.$$

Objective functions (1) and (2) are associated with the minimization of the makespan and the number of active pickers, respectively. Note that these two objective functions are usually conflicting, and our method that will be further discussed in the next section fixes the number of pickers and tackles the problem as a single objective MILP. Constraints (3) guarantee that each order must be assigned to exactly one batch. Constraints given in (4) ensure that the capacity of a trolley is respected. Note that, for each batch, at least one non-zero Y_b^o sets the value of W_b into one. Constraints (5) express that an active batch requires to be assigned to a picker. Moreover, assigning at least one batch to a picker makes his/her A_r equal to 1 (constraints (6)). In order to shrink constraints (6), we also propose a set of constraints that can be added to the model in (B.6) in Appendix B. However, the existing formulation meets all requirements. Constraints (7) and (8) cause exactly one departure from and one arrival to the depot for an active batch, respectively. Constraints (9) guarantee that a bin is not visited more than once in a batch tour. Constraints (10) play the role of flow conservation constraints in each batch. Constraints (11) ensure that for each SKU requested by an order assigned to a batch, we retrieve items – from one or multiple storage locations – as many as requested by that order. Constraints (12) guarantee that the sum of all units picked at v_i^k does not violate the inventory of this bin. In fact, this set of constraints coordinate pickers. Constraints (13) and (14) establish a relationship between the values of $X^b_{v_i^k v_j^p}$'s and Q_{ib}^{ko}'s. Note that $\sum_{o\in O} n_o^i$ behaves as a big M value in (13). Constraints (15) show that if a picker for batch b visits v_j^p immediately after v_i^k, the time between these two storage locations consists of two parts: 1- a TT between two bins ($\tau_{v_i^k v_j^p}$), and 2- a picking time at v_i^k ($T_{pick}(i)$). The constraints given in (15) also eliminate the solutions with at least one subtour from the feasible region. Constraints (16) define that a tour representing

a batch completes when its corresponding picker returns the depot. Constraints in (17) calculate the total RT of all (in)active pickers; we linearize these constraints in (B.5) in Appendix B. Constraints (18) ensure that the makespan is greater than or equal to the total RT of each picker. Constraints (19), (20), (21), and (22) specify the domains of the decision variables.

There are some solutions in the feasible region of [OPMSSS] which are equivalent due to the symmetries that exist in the problem. In Appendix B, we explain symmetry breaking constraints that can be added to [OPMSSS] and are practically useful for developing a solution for a wave OP process. Note that constraints given in (B.1)–(B.4) do not change the optimal solution of [OPMSSS] and only shrink the feasible region.

3. Methodology

[OPMSSS] is a bi-objective MILP, and hence, finding its non-dominated points or efficient solutions is of interest. An efficient solution for a multi-objective optimization program is a solution resulting in objective function values (a non-dominated point) that cannot be improved in all objective functions simultaneously (for basic definitions in multi-objective optimization, see Ehrgott (2000): Chapter 2). In [OPMSSS], we usually have a non-dominated point corresponding to each number of active pickers if the number of active pickers is not greater than the number of orders. In this section, assume that we consider [OPMSSS] only for minimizing the makespan deploying a certain number of active pickers; let $[\text{OPMSSS}]_{|R|}$ denote this single-objective program where $A_r := 1, \forall r \in R$. This MILP is highly complex such that commercial optimizers do not enable to find its optimal solution(s). Hence, we propose a heuristic to find an effective solution for a wave OP problem in a warehouse under the MSSS.

We address our heuristic as the Decomposition of OP into order Batching, batch Assignment, picker Routing subproblems (DOPBAR). Later on, for analyzing the impact of changing the number of pickers on the makespan, we reapply DOPBAR with another given number of pickers (i.e., DOPBAR for solving $[\text{OPMSSS}]_{|R|-1}$, $[\text{OPMSSS}]_{|R|-2}$, etc.). In order to present a clear picture of DOPBAR, we discuss about our algorithm in two levels: high-level description including the main steps of DOPBAR and detailed description containing the details of each step.

3.1. High-level description

First, we introduce the steps of our algorithm, then, we discuss about each step exploiting an example. DOPBAR includes the following steps for solving an instance of $[\text{OPMSSS}]_{|R|}$:

- Step 0 Generate a sort of orders (denoted by SoO).
- Step 1 Take the orders one by one based on the arrangement given in SoO, and for each order, dynamically select a number of storage locations that satisfy the requested items and update the left inventory level in those bins.
- Step 2 For each order $o \in O$, solve a TSP on the selected storage locations in Step 1 and the depot; and find solely its total RT.
- Step 3 Create a matrix of distances (M) between orders based on the savings that can be obtained by batching every two orders.
- Step 4 Vary the number of active batches we need to create ($|B|^A$) in a range (e.g., from a minimum possible value for the number of batches such as $|B|^{min}$ to a maximum value like $|B|^{max}$) and find the batching plan that results in the minimum makespan as follows:
  - Step 4.1 Batch orders into $|B|^A$ batches and find the RT of each batch.
  - Step 4.2 Given $|B|^A$ batches and their RTs obtained in Step 4.1, assign them to $|R|$ pickers for minimizing the makespan.
- Step 5 Generate a new SoO based on the results of Step 4. If the new SoO is repeated, terminate the algorithm and report the best solution found. Otherwise, go to Step 1.

We illustrate on these steps in simple terms using an example provided in Fig. 2 and a flowchart given in Fig. 3 corresponding to the example. In Fig. 2(a), "$*$ (#)" denotes "# number of unit(s) requested from SKU $*$". Note that the orders information and SLA given in Figs. 2(a) and 2(b) are repeated from previous examples (refer to Figs. A.1(a) and 1(b)).

Following Fig. 3, we show the implementation of the first iteration of the loop including from Step 0 to Step 5. Let $g = u = 1$ and $m = 3$ based on the warehouse setting given in Fig. 1(c), and $\tau_{pick} = 10$ to conduct calculations. Moreover, we assume that two active pickers exist. We proceed with the first loop as follows:

Step 0: We generate a SoO as 2, 4, 3, and 1.

Step 1: We take orders in the arrangement denoted by SoO and select some storage locations based on available inventories to satisfy each order (the selected bins are shown in Fig. 2(c), where each number written in a bin denotes the order that has a pick at that bin).

Step 2: We solve TSPs that correspond to collection of orders and find their RTs.

Step 3: We define M to assign a distance between every two orders (i.e., a larger RT saving by batching two orders gives a shorter distance between them).

Step 4: We apply a method based on the concept of the k-means clustering for batching orders into $|B|^A$ batches (Step 4.1) where $|B|^A \in \{|B|^{min}, \dots, |B|^{max}\}$, and assign them to two pickers (Step 4.2).

Step 5: Looking at the best OP plan found in Step 4, we update SoO to likely find a better solution in the next iteration.

Finally, we decide to either report the best solution found or continue with running the algorithm for another iteration.

In this example, we have assumed that batching any number of orders does not violate trolley capacity (Cap_{unit}). Thus, we can take advantage of the fact that when multiple orders are batched, the total RT of the batch is smaller than or equal to the sum of those orders' RTs.

As seen, the best makespan is possible based on $|B|^A = 2$ and equal to 76 time units. In Step 5, we generate a new SoO; e.g., 3, 1, 2, and 4. Then, we move to Step 1, consider orders based on the new arrangement, and select some bins for each of them. Fig. 2(d) shows a new set of selected bins for finding an OP plan in the second iteration of the loop (note that there is a bin common between Orders 1 and 2). We follow the loop of Step 1 to Step 5 for finding shorter makespan values. The loop terminates when we encounter a repeating SoO in Step 5. These steps are detailedly explained and integrated to establish a solid algorithm in Section 3.2.

3.2. Detailed description

We illustrate the implementation of all steps of DOPBAR in the following five subsections. In Section 3.2.6, we integrate all steps to establish our algorithm.

3.2.1. Step 0 and Step 1: for each order, find a small neighborhood of the storage locations

In this subsection, first, we elaborate on Step 1, and next, we explain how Step 0 is required to generate SoO for initiating Step 1. In Step 1 of DOPBAR, for each order, we look for a subset of the storage locations that can meet the items requested. Moreover, we aim to select these storage locations such that they are located in a small area and near each other. We present Fig. 4 to illustrate our rationale in finding a small neighborhood of the bins for collecting an order including 5(2),

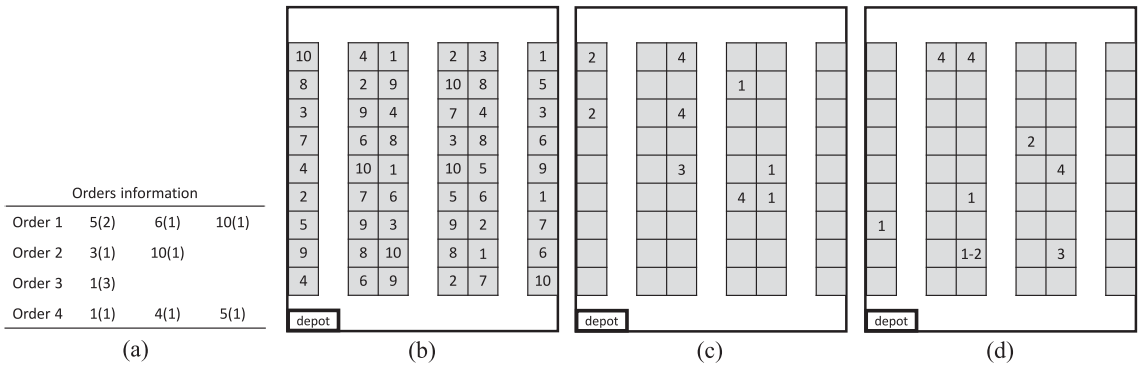


Fig. 2. An example for illustrating the steps of DOPBAR.

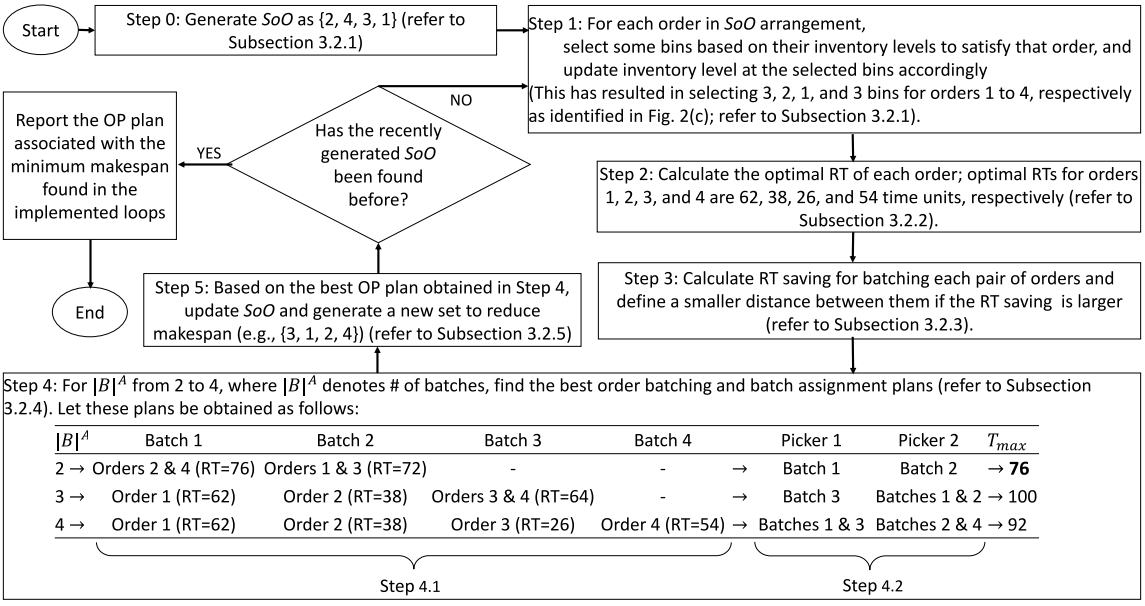


Fig. 3. A flowchart for presenting the steps of DOPBAR for the illustrative example.

6(1), and 10(1) (i.e., Order 1 which is discussed in an example in Fig. 1). We denote the selected storage locations by hatching their bins and the neighborhoods by blue polygons in Figs. 4(a)-4(c). Assume that each edge of a polygon associates with the TT value between its two corresponding bins. We express the two following statements for the reasoning behind our approach that is explained in this subsection:

First statement: Let Figs. 4(a) and 4(b) be of consideration; a smaller neighborhood significantly increases the probability of finding smaller TTs between the selected storage locations. We note that selecting storage locations to minimize the surface area of a polygon corresponding to them is complicated; instead, we aim to minimize pairwise TTs between every two selected bins.

Second statement: We need to consider the number of picks as every pick is a time consuming activity. As seen in Fig. 4(c), although the neighborhood is smaller than the neighborhood shown in Fig. 4(a) and the selected bins are closer to each other, the number of picks is four. Therefore, the selected bins in Fig. 4(a) may provide a faster retrieval than the selected bins in Fig. 4(c).

Assume that we have focused on meeting only one order (e.g., order $\hat{o}$). Hence, we apply the following components to build our mathematical model for finding a small neighborhood of the storage locations that can satisfy order $\hat{o}$ (i.e., v_i^k's where $i \in S_{\hat{o}}$ and $k \in I_i$):

Z_i^k: a decision variable that takes the value of 1 if v_i^k is selected to be in the subset of the storage locations we are interested in. Otherwise, it is equal to 0 (s.t. $i \in S_{\hat{o}}$ and $k \in I_i$); and

LCI: the list of current inventory level in the bins. For each $i \in I$ and $k \in I_i$, an element denoted by $\hat{q}_i^k$ exists in this list which shows the number of items available in storage location v_i^k. The initial value setting for these elements is as $\hat{q}_i^k := q_i^k$, $\forall i \in I$ and $k \in I_i$.

Assume that a set of storage locations exists which can satisfy order $\hat{o}$. We aim to select a subset of them that result in a small neighborhood considering the mentioned statements. Then, we select these storage locations such that a TT matrix with smaller values exists for them. In other words, the sum of the coefficients of the TT matrix for the selected storage locations becomes smaller. In order to capture such a subset of the storage locations, we solve a binary quadratic program (BQP), FSN($\hat{o}$, LCI), given in (23)–(25). We aim to find this subset of the storage locations such that they satisfy order $\hat{o}$ subject to the current available inventory in the storage locations. FSN($\hat{o}$, LCI) finds a small neighborhood of the storage locations by minimizing the sum of two expressions: 1- the sum of the TTs between every two selected storage locations (regarding the first statement), and 2- the time consumed on picking activities (regarding the second statement).

FSN($\hat{o}$, LCI):

$$\min \frac{1}{2} \sum_{i \in S_{\hat{o}}} \sum_{\substack{k \in I_i \\ \text{if } \hat{q}_i^k \neq 0}} \sum_{j \in S_{\hat{o}}} \sum_{\substack{p \in I_j \\ \text{if } \hat{q}_j^p \neq 0}} Z_i^k Z_j^p \tau_{v_i^k v_j^p} + \tau_{pick} \sum_{i \in S_{\hat{o}}} \sum_{\substack{k \in I_i \\ \text{if } \hat{q}_i^k \neq 0}} Z_i^k \tag{23}$$

$$\text{s.t.} \sum_{k \in I_i : \hat{q}_i^k \neq 0} \hat{q}_i^k Z_i^k \geq n_{\hat{o}}^i, \ \forall i \in S_{\hat{o}}, \tag{24}$$

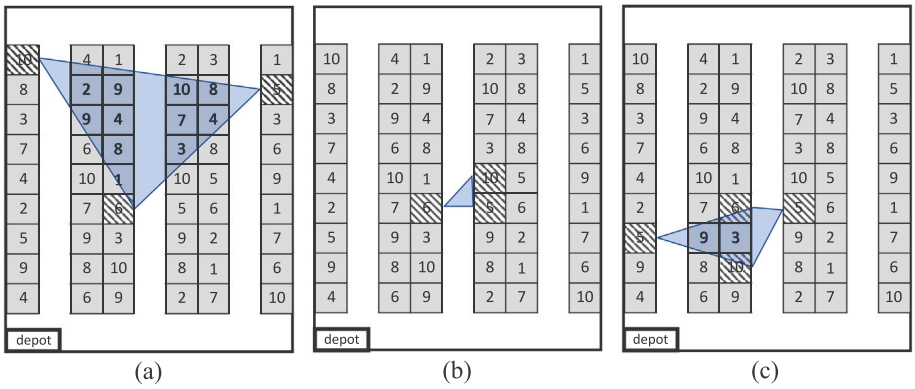


Fig. 4. An illustrative example to show the concept of finding a small neighborhood of multiple storage locations for satisfying the order of 5(2), 6(1), and 10(1).

$$Z_i^k \in \{0,1\},\ \forall i \in S_{\hat{o}}, k \in I_i \text{ if } \hat{q}_i^k \neq 0. \tag{25}$$

The objective function given in (23) includes two expressions as we mentioned. Note that a TT matrix is symmetric based on our assumptions, and hence, in the first term of the objective function, we apply coefficient $1/2$ to have the sum on the TTs between every two selected storage locations. Constraints (24) guarantee that for each ordered SKU, the inventory level in the selected storage locations satisfies order $\hat{o}$. Z_i^k's are binary decision variables and defined only for the requested SKUs subject to $\hat{q}_i^k \neq 0$ (constraints (25)). This formulation can be converted to a linear formulation using several methods (e.g., see Glover and Woolsey (1974) and Adams and Sherali (1986)). Then, the obtained binary linear program can be solved using known methods. Note that there are also methods for solving linearly constrained BQPs (Billionnet et al., 2005; Caprara, 2008; Gu and Chen, 2020).

Every number of orders must be batched together. When we follow the approach of selecting a small neighborhood of the bins for all orders, the obtained small neighborhoods can be potentially combined and result in shorter tours for retrieving each batch. We need to design a procedure to select a number of storage locations for each order individually. Procedure 1 lets us capture our objective in Step 1 by solving FSN formulation iteratively. This procedure uses the initial SLA, orders information, and the other parameters in $[\text{OPMSSS}]_{|R|}$ to generate two following outputs:

- For each order, the storage locations at which a pick exists, and the number of units that will be collected. We denote this output by set P_o for $o \in O$. Each element in P_o is shown by (v_i^k, Q_i^k) that denotes Q_i^k number of units must be collected at v_i^k.
- Consequently based on P_o, we define λ_{ik}^o as a parameter that takes the value of 1 if v_i^k is visited for retrieving the items ordered by o. Otherwise, its value is 0.

Since the orders are processed on an individual basis, we aim to consider larger orders earlier to let our algorithm have more options for these orders that can take longer to be retrieved. Thus, in line 1 of Procedure 1, we implement Step 0. We lexicographically sort the orders first based on the number of SKUs requested ($|S_o|$), then the total number of items ($\sum_{i\in I} n_o^i$), and then the largest number of units that are requested only for one SKU ($\max_{i\in I}\{n_o^i\}$). We denote the sorted order indices by SoO.

In line 2, Step 1 begins. We initialize list LCI, the set of selected storage locations (denoted by SSL), and λ_{ik}^o's. In the main loop of this procedure that begins at line 3, we select a subset of the storage locations for each order. Assume that $|S_{\hat{o}}| > 1$, then, we solve FSN$(\hat{o}, LCI)$ that results in some decision variables with optimal value equal to one (line 6); we update $\lambda_{ik}^{\hat{o}}$'s based on the optimal solution of these decision variables. For each of them that corresponds to a bin such as v_i^k, the number of units that must be collected is equal to the minimum of the available units and the requested units of SKU i (line 8). When we collect Q_i^k number of units for order $\hat{o}$, the following updates must be applied to (line 9):

- the number of unsatisfied units requested of SKU i; $n_{\hat{o}}^i := n_{\hat{o}}^i - Q_i^k$;
- the number of units left in bin v_i^k; $\hat{q}_i^k := \hat{q}_i^k - Q_i^k$ (i.e., LCI is updated);
- the set of selected storage locations (SSL); and
- set $P_{\hat{o}}$.

Now assume that $|S_{\hat{o}}| = 1$ (line 10). Then, we may enable to retrieve $\hat{o}$ from the storage locations that have been selected before. This approach allows us to visit a smaller number of storage locations in total and spend less time on picking activities. Let PSL be a set of prioritized storage locations that have been visited before, and each of them contains a sufficient number of units to meet $\hat{o}$ (line 11). If PSL is not an empty set (line 12), we find the nearest storage location to the depot among PSL's elements (line 13), and proceed with the updates in line 14. If PSL is an empty set, we redefine set PSL in line 16 as the set of all storage locations that contain a sufficient number of units to meet $\hat{o}$ (we eliminate the condition of $v_i^k \in SSL$). If such a storage location exists, we follow lines 13 and 14. Otherwise, it means that we cannot satisfy $\hat{o}$ by visiting only one storage location, and hence, we apply lines 6–9 where model FSN is solved.

Procedure 1 Select a subset of the storage locations for each order to find a small neighborhood.
Input: ( ), output: (P_o's, λ_{ik}^o's).

1: Sort all orders $o \in O$ lexicographically decreasing in terms of $|S_o|$, $\sum_{i\in I} n_o^i$, and $\max_{i\in I}\{n_o^i\}$; let SoO denote the result of sorting (Step 0 is provided by this line);
2: Initialize LCI; $SSL := \emptyset$; $\lambda_{ik}^o := 0$, $\forall o \in O, i \in I, k \in I_i$.
3: **for** each $\hat{o} \in SoO$, **do**
4: $P_{\hat{o}} := \emptyset$;
5: **if** $|S_{\hat{o}}| > 1$, **then**
6: solve FSN$(\hat{o}, LCI)$ and denote its solution by setting the values of $\lambda_{ik}^{\hat{o}}$'s into the optimal values of Z_i^k's;
7: **for** each $\lambda_{ik}^{\hat{o}}$ which is equal to 1, **do**
8: $Q_i^k := \min\{n_{\hat{o}}^i, \hat{q}_i^k\}$, ▷ The number of items collected at v_i^k for $\hat{o}$.
9: update $n_{\hat{o}}^i$, LCI, $SSL := SSL \cup \{v_i^k\}$, and $P_{\hat{o}} := P_{\hat{o}} \cup \{(v_i^k, Q_i^k)\}$;
10: **else**
11: $PSL := \{v_i^k | v_i^k \in SSL, \hat{q}_i^k \geq n_{\hat{o}}^i, i \in I, k \in I_i\}$;
12: **if** $|PSL| \geq 1$, **then**
13: $(\hat{i}, \hat{k}) := \underset{(i\in I, k\in I_i)}{\arg\min} \{\tau_{v_i^k v_{0e}^0} | v_i^k \in PSL\}$; $Q_{\hat{i}}^{\hat{k}} := n_{\hat{o}}^{\hat{i}}$; $\lambda_{\hat{i}\hat{k}}^{\hat{o}} := 1$.
14: Update $n_{\hat{o}}^{\hat{i}}$, LCI, $SSL := SSL \cup \{v_{\hat{i}}^{\hat{k}}\}$, and $P_{\hat{o}} := P_{\hat{o}} \cup \{(v_{\hat{i}}^{\hat{k}}, Q_{\hat{i}}^{\hat{k}})\}$;
15: **else**
16: $PSL := \{v_i^k | \hat{q}_i^k \geq n_{\hat{o}}^i, i \in I, k \in I_i\}$;
17: **if** $|PSL| \geq 1$, **then**
18: go to line 13;
19: **else**
20: go to line 6.
21: **return** P_o, $\forall o \in O$, and λ_{ik}^o, $\forall o \in O, i \in I, k \in I_i$.

In summary, in Step 1, we consider the orders one by one based on SoO generated in Step 0. For each order, we select some storage locations to satisfy that and update the remaining items in the bins. Meanwhile, for selecting those storage locations, we prefer to select the bins in which it has been decided to have a pick before.

One of considerable points in the implementation of Step 1 (Procedure 1), is the approach we use to select a subset of bins for an order. Our approach may not result in the minimum RT for retrieval of the order, and hence, can be questioned. In this regard, Weidinger (2018) and Daniels et al. (1998) propose a mathematical model to find the minimum TT for a single picker in collecting a number of units of multiple SKUs. Their model can be modified to capture the minimum RT for a single picker. We denote that model by SPR, and its detailed discussion is given in Appendix C. In Section 5, we analyze the effectiveness of solving FSN instances in Step 1.

3.2.2. *Step 2: for each order, find the shortest tour on the selected storage locations*

Thanks to Step 1, we have selected a number of storage locations for meeting each order and stored in P_o's. In Step 2, we solve a TSP on the selected storage locations for each order and the depot. There exist various (meta)heuristic algorithms for solving a TSP. In this paper, we solve OP instances in single-block rectangular warehouses, and hence, we find the exact optimal solutions for TSPs using the algorithm proposed by Ratliff and Rosenthal (1983). If a warehouse is not constructed based on a particular structure, we can apply other algorithms for the TSPs (e.g., Lin and Kernighan (1973) heuristic or its modifications). Then, let "TSPsolver" be a function that sorts the order of the elements in a P_o based on the solution of a TSP on the bins in P_o and the depot. We denote these sorted sets by P_o^{TSP}, $\forall o \in O$. For example, if $P_o^{TSP} = \{(v_1^1, Q_1^1), (v_3^2, Q_3^2), (v_2^1, Q_2^1)\}$, then the corresponding tour is $v_{0^s}^0$-v_1^1-v_3^2-v_2^1-$v_{0^e}^0$.

We apply Procedure 2 to solve the TSPs corresponding to the orders individually. For order o, P_o is sorted in line 2, and its RT, l^o, is calculated in line 3. This procedure returns l^o, $\forall o \in O$.

Procedure 2 Solve a TSP on the selected storage locations for each order.
Input: (P_o's), output: (l^o's).

1: **for** each $o \in O$, **do**
2: apply TSPsolver(P_o) and denote its result by P_o^{TSP};
3: calculate the tour RT (l^o) corresponding to P_o^{TSP};
4: **return** l^o, $\forall o \in O$.

3.2.3. *Step 3: create a matrix of distances for the orders*

Since we aim to apply a method similar to k-means clustering for order batching in Step 4.1, we need to define a distance between every pair of orders. We aim to calculate that by evaluating the RT savings of batching two orders together. When two tours associated with two orders are combined, all vertices that belong to either of them must be visited in the combined tour. Before explaining our remedy for calculating the distance between two orders, we state a claim: if two tours with at least one vertex in common exist, there is at least one route for their combined tour such that its length is not greater than the sum of both tours' minimum lengths.

This statement shows that we can possibly obtain savings in terms of total RT/TT when we batch orders (note that no saving exists in the worst case). Given orders o and o' with $l^o = 50$ and $l^{o'} = 30$ that are calculated in Procedure 2, we are certain that the shortest possible RT for a batch of both orders is not smaller than 50 ($= \max\{30, 50\}$). Hence, if the actual minimum RT of the batch of these orders is 50 ($l^{oo'} = 50$), we assign zero as the distance between these two orders ($M_{oo'} := 0$). Because the maximum possible RT saving for o' as the shorter-RT order has occurred. In fact, when o and o' are batched, for collecting o', a picker who is supposed to retrieve o, does not need to travel more nor spend more time on picking activities. In order to define a value as distance between two orders in general, we use the following formulation in Procedure 3 to create M:

$$M_{oo'} = \frac{l^{oo'} - \max\{l^o, l^{o'}\}}{l^o + l^{o'}}.$$

In lines 1 and 2, two loops start to consider pairs of orders. Note that matrix M is symmetric, and hence, we add condition $o' \geq o$ in line 2. For the sake of consistency, we set diagonal elements in the matrix to zero (line 4). In line 6, we calculate our defined distance between orders o and o'. Note that the formula may report a negative value if someone uses a non-exact method to solve the TSPs corresponding to each retrieval. In that case, we can simply assume that $M_{oo'} = \max\{0, \frac{l^{oo'} - \max\{l^o, l^{o'}\}}{l^o + l^{o'}}\}$.

Procedure 3 Create a matrix of distances for orders.
Input: (l^o's), output: (M).

1: **for** each $o \in O$, **do**
2: **for** each $o' \in O$ where $o' \geq o$, **do**
3: **if** $o' = o$, **then**
4: $M_{oo'} := 0$;
5: **else**
6: $M_{oo'}$ and $M_{o'o} := \frac{l^{oo'} - \max\{l^o, l^{o'}\}}{l^o + l^{o'}}$;
7: **return** M.

3.2.4. *Step 4: order batching and batch assignment*

In Step 1 and Step 2, we decided about the storage locations at which orders will be collected. In Step 3, we evaluated the proximity between every two orders and created matrix M. In this subsection, we propose a method for order batching deploying an approach similar to the k-means clustering (Step 4.1). Then, we use that batching plan to determine about batch assignment (Step 4.2).

K-means clustering is a tool for grouping a number of objects to a certain number of clusters based on the distances between the objects. In the k-means clustering (Lloyd, 1982), at each iteration, a centroid for each cluster is identified. Then, a subset of objects is assigned to each centroid based on their proximities. Each subset of objects is a cluster indeed, and we compute the centroid of that cluster. The k-means clustering iteratively finds new clusters and consequently their centroids until the convergence of centroids. We customize this method for batching/clustering orders. We assume that at each iteration, a centroid denoted by C^b is an order ($b \in B^A$ where $B^A := \{1, \ldots, |B|^A\}$). We require to assign a subset of orders to each centroid to create $|B|^A$ clusters in total. On the other hand, we aim to give a larger weight to the orders with a larger number of picking storage locations. Hence, we define $\phi_o := \sum_{i \in I} \sum_{k \in I_i} \lambda_{ik}^o$ as order o's weight $\forall o \in O$. In order to assign orders to the centroids, the following binary programming problem is solved:

$$\text{ASG}(M, \phi_o\text{'s}, B^A):$$

$$\min \sum_{b \in B^A} \sum_{o \in O} Y_b^o M_{oC^b} \phi_o \tag{26}$$

$$\text{s.t.} \sum_{o \in O} \sum_{i \in I} n_o^i Y_b^o \leq Cap_{unit}, \ \forall b \in B^A, \tag{27}$$

$$\sum_{b \in B^A} Y_b^o = 1, \ \forall o \in O, \tag{28}$$

$$Y_b^o \in \{0, 1\}, \ \forall b \in B^A, o \in O, \tag{29}$$

where (26) is the weighted-sum of distances between the orders of a cluster and its centroid. Constraints (27), (28), and (29) correspond to constraints (4), (3), and (20) given in [OPMSSS], respectively. Note that the optimal values of Y_b^o's identify the orders that are to assigned to cluster b based on the existing centroids.

When an order batching plan is determined, the RT of each batch can be calculated (refer to Step 2 and TSPsolver in Section 3.2.2). Let η^b be the RT of batch $b \in B^A$. Therefore, assigning the created batches to a certain number of pickers such as $|R|$ is the same as unrelated identical parallel machine scheduling (UPMS). In the UPMS problem

that is equivalent to our batch assignment problem, we minimize the makespan by assigning $|B|^A$ jobs with η^b processing time for job b to $|R|$ machines. This problem exactly represents batch assignment and is formulated as follows:

UPMS(η^b's, B^A) :

$$\min\ T_{max} \tag{30}$$

$$\text{s.t.} \sum_{b\in B^A} \eta^b U_b^r = \bar{T}^r,\ \forall r \in R, \tag{31}$$

$$\sum_{r\in R} U_b^r = 1,\ \forall b \in B^A, \tag{32}$$

$$\bar{T}^r \le T_{max}, \forall r \in R, \tag{33}$$

$$\bar{T}^r, T_{max} \in \mathbb{R}^+ \cup \{0\},\ \forall r \in R, \tag{34}$$

$$U_b^r \in \{0,1\}, \forall b \in B^A, r \in R, \tag{35}$$

where (30), (31), (32), (33), (34), and (35) correspond to (1), (17), (5), (18), (22), and (20) in [OPMSSS], respectively.

The UPMS problems are well studied in the field of operations research, and a number of survey studies are available to cover different assumptions and algorithms in this area (Pfund et al., 2004). Various algorithms are proposed to obtain effective results for the classic UPMS (e.g., Lin et al. (2011) and Lenstra et al. (1990)). However, for the UPMS instances need to be solved for solving an $[\text{OPMSSS}]_{|R|}$, a commercial optimizer can reach 0.5% optimality gap in a few seconds of CPU time. Thus, we use a commercial solver for this purpose. If the solver is not suitable for solving emerging UPMS instances, the proposed methods in the literature can be applied.

Next, we combine our customized k-means clustering and batch assignment approach to establish Procedure 4 for implementing Step 4. We aim to know if order o is in batch b (denoted by $\gamma_b^o = 1$) and batch b is assigned to picker r (denoted by $\theta_b^r = 1$). We initiate this procedure in line 1. Note that the maximum number of batches cannot exceed the number of orders. Steps 4.1 and 4.2 are implemented within lines 3–12 and line 13, respectively. We solve an instance of ASG in line 6. If that instance is infeasible, it means that the number of batches, $|B|^A$, does not suffice to create clusters without violating Cap_{unit}. Hence, we go to line 2 and increase the number of active batches (line 8). Based on ASG's optimal solution, we find a new centroid for each batch and update C^b's. For this purpose, first, we find the *center of gravity* of batch b in line 10 (the center of gravity of a set of bins is a storage location that sum of its distances from those bins is minimum). Then, in line 11, we find the order that sum of distances between its picking storage locations (i.e., bins with λ_{ik}^o equal to 1) and the cluster's center of gravity is minimum. We also include the weight of that order (ϕ_o) in the formulation as a larger order with many picking bins has a chance to be selected as the centroid of a cluster. In lines 2–13, the number of active pickers is constant, and we select the solution that gives the minimum makespan in line 14.

In this subsection, we proposed a procedure to batch orders and assign batches to pickers when storage locations for picking are predetermined. There are other algorithms in the literature suitable for this purpose. Those algorithms or their customized versions can be integrated into DOPBAR. However, in Section 5 (Numerical Experiments), we show that our proposed implementation for the steps of our algorithm establishes a solid design which efficiently and effectively solves [OPMSSS] instances.

3.2.5. *Step 5: update SoO based on the obtained solution in Step 4*

The solution found in Step 4 is influenced by SoO found in Step 0. There is no guarantee to find the best makespan applying Step 1–Step 4 by SoO obtained in line 1 of Procedure 1. Therefore, in Step 5, we rearrange SoO in a way that may cause the makespan reduced in the next iteration.

The main rationale is to focus on more time-consuming orders and select some storage locations for them earlier. We clarify this point through an example. Given two pickers and seven orders, assume that Step 0–Step 4 have been compiled once, and the following OP plan is produced:

- Picker 1:
  - Batch 1 takes 85 time units and includes Order 1 with 3 SKUs and Order 5 with 2 SKUs;
  - Batch 2 takes 105 time units and includes Order 4 with 2 SKUs and Order 6 with 1 SKU.
- Picker 2:
  - Batch 3 takes 98 time units and includes Order 2 with 2 SKUs;
  - Batch 4 takes 120 time units and includes Order 3 with 3 SKUs and Order 7 with 4 SKUs.

We take the orders assigned to Picker 2 who has a longer RT (218 time units that is greater than 190 time units). We assume that $120\times\frac{4}{4+3}$ time units are averagely required to collect Order 7. This value can be calculated for Orders 2 and 3 assigned to Picker 2. Then, we prioritize orders with larger average collection times (ACT) in the new SoO.

Procedure 5 shows how to implement Step 5. In line 2, we take pickers based on their descending RTs. Then, for an order assigned to a batch that belongs to picker r, we calculate ACT value in line 5. We gradually create a new SoO in line 6 and return it in line 7.

Procedure 4 Order batching and batch assignment for a set of predetermined storage locations.
Input: (λ_{ik}^o's, M), output: (γ_b^o's, θ_b^r's, η^b's).

1: $\phi_o := \sum_{i\in I}\sum_{k\in I_i} \lambda_{ik}^o,\ \forall o \in O;\ |B|^{min} := \max\{|R|, \lfloor \frac{\sum_{i\in I}\sum_{o\in O} n_o^i}{Cap_{unit}} \rfloor\};\ |B|^{max} := |O|;$
2: **for** $|B|^A$ from $|B|^{min}$ to $|B|^{max}$, **do**
3: $B^A := \{1, \dots, |B|^A\}$.
4: Select $|B|^A$ orders randomly as the centroids denoted by C^b ($b \in B^A$);
5: **while** the new set of centroids are not converged to the previous centroid set **do**
6: solve ASG(M, ϕ_o's, B^A) and denote its optimal solution for variable Y_b^o by γ_b^o.
7: **if** ASG(M, ϕ_o's, B^A) is infeasible **then**
8: go to line 2 and add one to the current number of active batches.
9: **for** $b \in B^A$, **do**
10: $(\hat{j}, \hat{p}) := \underset{j\in I, p\in I_j}{\arg\min} \{\sum_{i\in I}\sum_{k\in I_i} ((\sum_{o\in O \text{ if } \gamma_b^o=1} \lambda_{ik}^o) \times \tau_{v_i^k v_j^p})\}$. ▷ cluster b's center of gravity
11: C^b is updated as $C^b := \underset{o\in O \text{ if } \gamma_b^o=1}{\arg\min} \{\frac{\sum_{i\in I}\sum_{k\in I_i} \lambda_{ik}^o \tau_{v_i^k v_{\hat{j}}}}{\phi_o}\}$.
12: Using TSPsolver, find η^b as the RT of batch b, $\forall b \in B^A$.▷ TSPsolver is discussed in Section 3.2.2
13: Solve UPMS(η^b's, B^A) and find the minimum makespan using $|B|^A$ batches; denote the optimal value for U_b^r by θ_b^r;
14: **return** the values of θ_b^r's and γ_b^o's associated with a specific $|B|^A$ that result in the minimum makespan and η^b's ($\forall b \in B^A$, $r \in R$, and $o \in O$). ▷ the best makespan within the loop in lines 2–13 must be captured.

Procedure 5 Generate a new SoO based on the result obtained in Step 4.
Input: (γ_b^o's, θ_b^r's, η^b's), output: (SoO).

1: Sort pickers based on the longest RT to the shortest RT (picker r's RT is equal to $\sum_{b\in B^A} \theta_b^r \eta^b$).
2: **for** each picker $r \in R$ (descending RT), **do**
3: **for** each batch $b \in B^A$ where $\theta_b^r = 1$, **do**
4: **for** each order $o \in O$ where $\gamma_b^o = 1$, **do**
5: $\text{ACT}_o = \eta^b \times \frac{|S_o|}{\sum_{o\in O}(\gamma_b^o |S_o|)}$.
6: With a larger ACT_o, give a higher priority to its corresponding order in SoO.
7: **return** new SoO.

3.2.6. *Integrate Procedures 1–5 for solving* $[\text{OPMSSS}]_{|R|}$

We aim to integrate the procedures explained in Sections 3.2.1–3.2.5 to find a solution for an OP planning problem under the MSSS.

DOPBAR is presented in Algorithm 1 and solves an instance of $[\text{OPMSSS}]_{|R|}$ by generating an effective OP plan.

We set a maximum number for iterating DOPBAR denoted by $MaxIter$. In fact, Step 0 generates SoO to initiate Step 1–Step 4 in the first iteration. Then, for iterating the algorithm more than once, Step 5 regenerates a new SoO for Step 1–Step 4.

In line 2 of Algorithm 1, we terminate DOPBAR when either of the following conditions is met: 1- the number of iterations reaches $MaxIter$, and 2- a repeated SoO in Step 5—in line 10—appears. When we are in the first iteration, Procedure 1 is applied. In lines 7–9, we execute Procedures 2, 3, and 4 for finding a solution for our problem instance. Next, we use Procedure 5 for finding a new sort of orders that may improve the solution which will be obtained in the next iteration. Note that in the second iteration, we skip the first line of Procedure 1 (i.e., Step 0) and continue from its second line (line 6). If we find a SoO in line 10 that has been explored before, the algorithm is terminated. Finally, in line 12, we report the OP plan that results in the minimum makespan (note that we find various makespan values during the while loop within lines 2–11). Actually, a practical OP plan includes:

- the storage locations that must be visited for each order and the number of units that must be picked at each bin (P_o's),
- batching plan to identify the orders that need to be batched together (γ_b^o's), and
- batch assignment plan for assigning the batches to the pickers available (θ_b^r's).

Algorithm 1 Find a solution for an OP process under the MSSS (DOPBAR for solving $[\text{OPMSSS}]_{|R|}$).
Input: ( ), output: (an OP plan).

1: $Iter := 1$;
2: **while** $Iter \leq MaxIter$ and SoO not repeated **do**
3: **if** $Iter = 1$ **then**
4: Procedure 1( ) $\Rightarrow$ (P_o's, λ_{ik}^o's).
5: **else**
6: Skip the first line of Procedure 1; and compile Procedure 1( ) $\Rightarrow$ (P_o's, λ_{ik}^o's).
7: Procedure 2(P_o's) $\Rightarrow$ (l^o's).
8: Procedure 3(l^o's) $\Rightarrow$ (M).
9: Procedure 4(λ_{ik}^o's, M) $\Rightarrow$ (γ_b^o's, θ_b^r's, η^b's). ▷ T_{max} can be calculated using TSPsolver
10: Procedure 5(γ_b^o's, θ_b^r's, η^b's) $\Rightarrow$ (SoO).
11: $Iter := Iter + 1$;
12: **return** the best OP plan with the minimum makespan obtained in the while loop (an OP plan can be produced by the values of γ_b^o's and θ_b^r's and sets P_o's).

We aim to remark another point related to TSPsolver. This function basically creates Procedure 2. However, TSPsolver is required for implementing other steps/procedures as well; e.g., in calculating the RT of two/multiple combined orders in line 6 of Procedure 3 and line 12 of Procedure 4.

4. Generating problem instances

In this section, we introduce various problem settings for evaluating our method. Note that the applied settings are basically either suggested or inspired by different studies such as Theys et al. (2010), Van Gils et al. (2019a), and Weidinger (2018). In terms of the layout of a rectangular single-level warehouse with single-block (we consider single-block warehouses in this paper to stay focused on OP planning under the MSSS instead of the impact of having multiple blocks). When a picker moves between two storage locations, s/he travels between the picking nodes that correspond to those storage locations passing through some edges. An example for those picking nodes and edges are shown in Fig. 1(c). Let all picking nodes be located in the middle of their corresponding aisles. We set the widths of the picking aisles, cross aisles, bins, and the average velocity of a picker to 1.5 m, 5 m, 0.75 meter, and 0.9 meter/second, respectively. We assume that the depth of a bin is equal to its width, and then, $g = 0.75/0.9 \approx 0.85$, $m = 2g +$ picking aisle width ≈ 3.35, and $u = \frac{1}{2}(g +$ cross aisle width$) \approx 3.2$ seconds (remind what g, m, and u refer to in Fig. 1(c)). Note that we have rounded the values for simplifying the values in the TT matrix of the storage locations. We also set the value of τ_{pick} to 30 s and Cap_{unit} to 25.

In terms of the products stored in a warehouse, in many cases, a Pareto categorization exists. Thus, we assume that a class-based categorization exists such that 20%, 30%, and 50% of the SKUs are categorized into A-class, B-class, and C-class products, respectively. Each bin contains a number of units of an A-class product with the probability of 80%. The probabilities of having B-class and C-class products in a bin are 15% and 5%, respectively.

By generating several groups of problem instances, we aim to evaluate various SLA strategies in response to different orders for wave OP planning. When the DSS applies, we generate a SLA in two ways which are recommended in the literature to have shorter routes: across-aisle class-based (ACDSS) and within-aisle class-based (WCDSS). Regarding these DSS-based warehouses, note that there may exist multiple bins which are next to each other for each SKU (e.g., Fig. 1(a)). When the MSSS applies, a scattering degree (β, where $0 \leq \beta \leq 1$) is required for generating a SLA instance. Let the RMSSS-β specify a randomized SLA under the MSSS with β level of scattering. Weidinger (2018) generates a RMSSS-β-based SLA instance based on the number of clusters. A cluster is defined as one bin or a number of adjacent storage locations that contain the same SKU (see Fig. 5(a) where bins with diagonal stripes are adjacent to the black storage location and bins with vertical stripes are adjacent for the white storage location). In their definition, the scattering degree increases by the number of clusters, and two following drawbacks exist:

- *The proximity of the clusters of the same SKU* is not considered. I.e., the bins which are not adjacent but close to each other cannot be taken into account. E.g., in both Figs. 5(b) and 5(c), four clusters for SKU 1 exist; however, the SLA given in Fig. 5(c) is less scattered in terms of SKU 1.
- *The number of units in each bin* is not regarded. E.g., in both Figs. 5(d) and 5(e), two clusters for SKU 1 exist; however, we can intuitively say that the SLA given in Fig. 5(e) is less scattered in terms of SKU 1.

In order to address the proximity of the clusters that correspond to the same SKU and the number of units in each bin, we generate RMSSS-β-based instances using a different approach. In Fig. 6(a), the bins numbered as 1 to 6 show the vacant bins. Assume that the other bins are occupied by some items and the bins denoted by 1(1), 1(2), and 1(3) are the only storage locations that contain SKU 1 (the numbers in parentheses denote the number of units). Assume that we are in a stage in which we require to assign some units of SKU 1 to one vacant bin considering scattering degree 0.6 (i.e., $\beta = 0.6$). In Fig. 6(b), the TT matrix from the bins containing SKU 1 to the vacant bins is presented. We need to select one of the vacant bins considering the aforementioned issues. Thus, we calculate a weighted-sum TT from the bins containing SKU 1 to each vacant storage location (Fig. 6(c)). For example, this weighted-sum TT for vacant bin 3 is equal to $15.7 \times 1 + 14 \times 3 + 10.6 \times 2 = 78.9$. This value shows how much assigning a number of units of SKU 1 to vacant bin 3 causes scattering respect to the bins that contain SKU 1. In Fig. 6(c), a range of values is presented with maximum value equal to 96.1 and minimum value equal to 33.5. When $\beta = 0.6$, the ideal weighted-sum TT is $33.5 + \beta(96.1 - 33.5) = 69.02$. Therefore, we select the vacant bin that has the closest weighted-sum TT to 69.02; and assign SKU 1 to vacant bin 3. Applying this approach, we signify "the number of units in each bin" by using this number as a weight in the weighted-sum TT (e.g., if many units of SKU 1 exist in one bin, being far from that bin is more important in terms of scattering). In addition, "the proximity of the clusters of the same SKU" is addressed by using TTs. We provide a complete discussion for generating a SLA

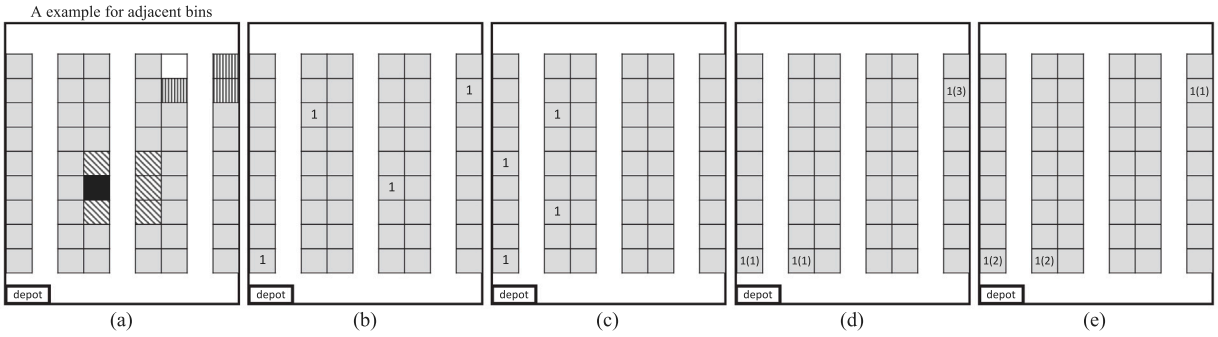
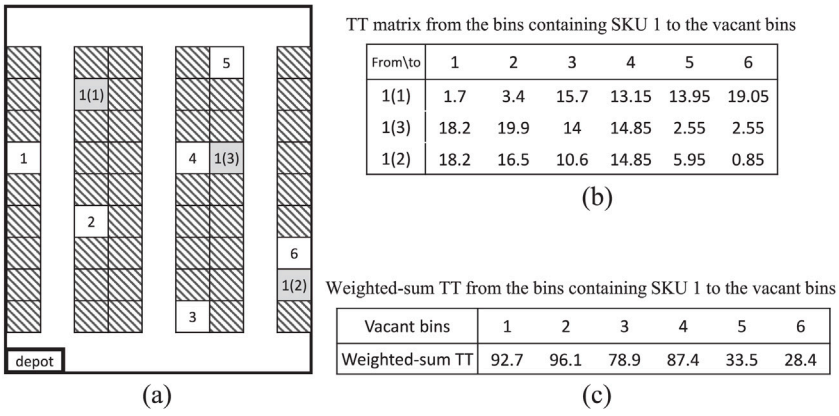


Fig. 5. An illustration on the issues that cannot be considered by the number of clusters as a criterion for the scattering degree.



From\to	1	2	3	4	5	6
1(1)	1.7	3.4	15.7	13.15	13.95	19.05
1(3)	18.2	19.9	14	14.85	2.55	2.55
1(2)	18.2	16.5	10.6	14.85	5.95	0.85

Vacant bins	1	2	3	4	5	6
Weighted-sum TT	92.7	96.1	78.9	87.4	33.5	28.4

Fig. 6. An illustration on our approach for selecting one vacant bin for SKU 1 based on a certain scattering degree.

instance based on the RMSSS-β, the mixture of the across-aisle and the MSSS (ACMSSS), and the combination of the within-aisle and the MSSS (WCMSSS) in Appendix D.

In order to test DOPBAR's effectiveness, we generate instances in four different sizes. We refer to these instance sizes as test, small, medium, and large-sized instances. Note that "test-sized" instances are sufficiently small-sized instances that will be used for making a comparison between our solutions and the solutions obtained by a commercial optimizer. Table 3 shows the parameters' setting for each set of instances ($Cap_{unit} = 9$ for test-sized instances). We solve instances corresponding to the warehouses of 750, 2400, and 8400 square meters. Weidinger (2018) states that these warehouse sizes can represent real e-commerce warehouses which exist for online retailing. In order to generate diverse SLA instances based on various SLA strategies, we apply eight different SLA strategies.

For diversifying order instances, we require to set order-related parameters. We determine the maximum number of SKUs that may be requested in one order ($MaxSKU$). We set this number to either two or six for small, medium, and large-sized instances. For either of cases, the probability distribution function of the number of SKUs in one order is uniform (i.e., 1/2 or 1/6). Moreover, we assume that when a customer aims to buy a SKU, s/he asks for at most two or four units (in all instance types except the test-sized instances). For either of cases, a probability distribution function is given in Table 3. Note that requesting a larger number of units of the same SKU is less probable in an e-commerce market. Moreover, as we need to stay focused on businesses in which a form of the MSSS is suitable, we do not generate instances with very different profiles. For those kinds of order profiles, more investigations are required to examine the efficiency of the MSSS-based warehouses and evaluate our proposed method.

We also change the number of active pickers to examine the influences of the workforce size on the makespan. We solve small, medium, and large-sized instances with three (i.e., 3, 4, and 5), four (i.e., 4, 5, 6, and 7), and five (i.e., 6, 7, 8, 9, and 10) different values as the number of active pickers, respectively.

According to the changing parameters, we generate order instances based on four different scenarios regardless of the number of orders. For each scenario, we randomly generate 12 instances. We examine the performance of each SLA strategy with a given workforce level in responding to 48 order instances. Therefore, we solve $8 \times 48 = 384$ instances for each problems size when the workforce level is fixed; i.e., in total, 4608 ($= 384 \times (3+4+5)$) instances excluding the test-sized instances are solved.

5. Numerical experiments

In this section, we report our numerical results after implementing DOPBAR in Python 3.7 and Gurobi 8.1.1 optimizer on a PC with a 64-bit Windows 10 operating system, core i7-8550U processor at 1.80 GHz, and with 16.0 GB of RAM. We discuss about DOPBAR's time performance and solutions' quality in the following subsections. We mainly report CPU times, makespan values, and total TTs all in second.

5.1. DOPBAR versus Gurobi

First, we aim to evaluate the quality of the solutions obtained by DOPBAR comparing with the solutions that can be obtained by Gurobi optimizer. An instance of $[\text{OPMSSS}]_{|R|}$ is highly complex, and commercial optimizers cannot find a feasible solution for the small/medium/large-sized instances of this problem in a reasonable cut-off time such as one hour. Then, we evaluate the performance of DOPBAR by comparing the makespan values obtained by this algorithm for the test-sized instances versus Gurobi optimizer's (detailed parameter setting is given in Table 3). Table 4 shows a comparison between DOPBAR and Gurobi optimizer in solving 384 instances of $[\text{OPMSSS}]_3$. We set the $MaxIter$ for DOPBAR into 30 and a 60-minute time limit

Table 3
Parameters' setting for layout and order-related attributes in test, small, medium, and large-sized instances.

Layout and storage-related				
Attributes/parameters	Test	Small	Medium	Large
# of aisles	4	10	20	40
# of bins in one aisle	16	40	80	160
Equivalent warehouse space area in sq.m	192	750	2400	8400
# of SKUs ($\|I\|$)	16	80	200	800
Maximum # of units that can be placed in a bin ($\bar{q}$)	3	3	3	3
SLA strategies	ACDSS, WCDSS, ACMSSS, WCMSSS, RMSSS-0.25, RMSSS-0.5, RMSSS-0.75, RMSSS-1			
Order-related				
Attributes/parameters	Test	Small	Medium	Large
# of orders ($\|O\|$)	10	50	150	250
Maximum # of SKUs that may be requested by one order ($MaxSKU$)	3 or 2	6 or 2	6 or 2	6 or 2
$\max_{i\in I, o\in O}\{n_o^i\}$ ($MaxI$)*	3 or 2	4 or 2	4 or 2	4 or 2
* Probability distribution function for the # of units that are requested from one SKU in one order				
Given $MaxI = 2$ and $n_o^i > 0$,	$p(n_o^i = 1) = 0.7,\ p(n_o^i = 2) = 0.3$			
Given $MaxI = 3$ and $n_o^i > 0$,	$p(n_o^i = 1) = 0.6,\ p(n_o^i = 2) = 0.3,\ p(n_o^i = 3) = 0.1$			
Given $MaxI = 4$ and $n_o^i > 0$,	$p(n_o^i = 1) = 0.5,\ p(n_o^i = 2) = 0.3,\ p(n_o^i = 3) = 0.15,\ p(n_o^i = 4) = 0.05$			
Picker-related				
Attributes/parameters	Test	Small	Medium	Large
Range of the # of active pickers	$\{3\}$	$\{3, 4, 5\}$	$\{4, \ldots, 7\}$	$\{6, \ldots, 10\}$

Table 4
Comparison between the makespan values obtained by Gurobi optimizer with a 60-minute time limit and DOPBAR in solving 384 test-sized instances of $[OPMSSS]_3$ and $[OPDSS]_3$.

Changing factor		# of inst.	[OPMSSS]				[OPDSS]			
			DOPBAR CPUT	T_{max}	Gurobi gap	DOPBAR deviation	DOPBAR CPUT	T_{max}	Gurobi gap	DOPBAR deviation
SLAs	ACDSS	48	69.9	235.4	25.64%	−6.74%	3.6	249	16.14%	−5.84%
	WCDSS	48	78.6	248.1	28.20%	−7.03%	3.8	263.2	17.82%	−5.57%
	ACMSSS	48	52.7	227.3	23.67%	−4.16%	3.4	246	14.85%	−3.15%
	WCMSSS	48	65.0	236.7	24.50%	−5.00%	3.3	247.5	15.95%	−3.94%
	RMSSS-0.25	48	61.3	226	24.07%	−5.26%	3.4	242.6	15.08%	−3.61%
	RMSSS-0.5	48	52.1	221.5	25.55%	−3.02%	3.1	238.3	15.60%	−2.47%
	RMSSS-0.75	48	54.4	229.3	25.50%	−2.76%	3.4	246.5	16.94%	−0.45%
	RMSSS-1	48	71.8	227.6	25.78%	−5.55%	3.3	244.5	16.78%	−2.95%
Orders	(2, 2)	96	38.8	186.7	20.32%	−8.89%	2.5	203.5	10.40%	−7.32%
	(2, 3)	96	57.3	192.3	22.57%	−6.44%	3.3	208.2	12.62%	−4.88%
	(3, 2)	96	72.9	262.9	28.16%	−5.06%	4.0	276.1	20.19%	−3.34%
	(3, 3)	96	83.8	284	30.40%	0.63%	3.9	301.1	21.37%	1.54%
Total average			63.2	231.49	25.36%	−4.94%	3.4	247.21	16.14%	−3.50%

on the CPU time (CPUT) for solving each instance using Gurobi (a 60-minute limit is a large cut-off time for such a small instance of a warehouse). We report the results by changing the SLA strategy and orders' setting. Note that a pair of numbers in parentheses refer to $MaxSKU$ and $MaxI$; e.g., (2, 3) denotes $MaxSKU = 2$ and $MaxI = 3$. The third column of this table shows the number of solved instances that correspond to a certain SLA strategy or setting of orders.

Columns 4 to 7 report the average CPUT of DOPBAR (in second), average makespan obtained by our algorithm (in second), Gurobi optimality gap after 1 h, and the deviation of the makespan found by DOPBAR from the makespan that Gurobi calculates in an hour (a negative and positive value indicate that DOPBAR has found a larger and smaller makespan value, respectively). DOPBAR solves test-sized instances in 63.2 s in average. Moreover, in terms of the makespan values, it averagely results in −4.94% deviation from Gurobi's nearly optimal solutions. By exploring the optimality gaps, increasing either of $MaxSKU$ and $MaxI$ influences on the difficulty of an instance.

Regarding columns 8 to 11, we aim to show the effectiveness of our approach if the storage locations that must be visited are predetermined. In this case, our OP planning under the MSSS becomes OP planning under the DSS (i.e., [OPMSSS] and $[OPMSSS]_{|R|}$ are converted to [OPDSS] and $[OPDSS]_{|R|}$ which are formulated in Appendix E). As a subset of the storage locations are selected, DOPBAR does not iterate and operates only for one iteration. We show that DOPBAR averagely in 3.4 s finds a makespan that deviates −3.5% from the solution of Gurobi in an hour. In overall, Table 4 demonstrates that DOPBAR results in an effective performance in solving $[OPMSSS]_{|R|}$ and $[OPDSS]_{|R|}$ instances. Moreover, DOPBAR outperforms Gurobi optimizer in terms of solution quality when (3, 3) order setting applies.

We aim to briefly discuss about Gurobi optimality gap values in Table 4 and consequently the solution quality obtained by Gurobi in one hour. In solving $[OPMSSS]_{|R|}$ and $[OPDSS]_{|R|}$ instances, gap values decrease based on the improvements of global primal bound and global dual bound. We have observed that gap value while solving an instance using Gurobi decreases considerably slowly after approximately 10 min of starting optimization. Moreover, we do not observe a significant reduce in objective function values after a time duration like 10 min. This is a usual behavior of gap and objective function values in solving many NP-hard problems. In fact, the solver may reach a global primal bound that is actual optimal solution, but the optimality gap is greater than zero because a tight global dual bound has not been obtained yet. Gurobi needs time to substantially improve the global dual bound and close the gap. Thus, our solutions obtained by Gurobi optimizer after one hour can be (nearly) optimal. We briefly present the related evidences in a figure given in Appendix F.

Table 5
Numerical results in solving the small-sized instances of $[OPMSSS]_{|R|}$ (i.e., $|R|$ varies from 3 to 5).

# of pickers	Criteria	ACDSS	WCDSS	ACMSSS	WCMSSS	RMSSS-0.25	RMSSS-0.5	RMSSS-0.75	RMSSS-1
5	T_{max}	1091	1079	**951**	977	957	**942**	**940**	959
	Sum of total TTs	1695	1617	**1081**	1212	1139	1128	**1118**	**1117**
	CPUT	153.4	146	84.5	89.1	89.3	91.6	94.9	93.7
4	T_{max}	1338	1318	1172	1194	1170	**1144**	**1148**	**1164**
	Sum of total TTs	1687	1604	**1121**	1194	1132	**1113**	1121	**1100**
	CPUT	153.6	145.2	84.2	89.1	88.9	91.6	94.7	93.5
3	T_{max}	1747	1721	1530	1552	1529	**1500**	**1501**	**1518**
	Sum of total TTs	1620	1539	**1060**	1130	1104	**1059**	**1061**	1063
	CPUT	152.1	143.9	82.8	87.6	87.8	90.3	93.3	92.2
Freq. among best three		0	0	4	0	0	5	5	4
Gurobi gap in FSN		0%	0%	0%	0%	0%	0%	0%	0%

Table 6
Numerical results in solving the medium-sized instances of $[OPMSSS]_{|R|}$ (i.e., $|R|$ varies from 4 to 7).

# of pickers	Criteria	ACDSS	WCDSS	ACMSSS	WCMSSS	RMSSS-0.25	RMSSS-0.5	RMSSS-0.75	RMSSS-1
7	T_{max}	2780	2704	**2174**	2282	2203	2202	**2191**	**2191**
	Sum of total TTs	8470	8000	**4732**	5409	4875	4884	**4800**	**4800**
	CPUT	575.8	561	274	322.1	273.3	288	309.7	316.9
6	T_{max}	3235	3152	**2531**	2655	2559	2557	**2549**	**2545**
	Sum of total TTs	8393	8028	**4688**	5344	4816	4834	**4778**	**4746**
	CPUT	587.7	570	280.7	320.7	267.3	285.5	304.8	299.9
5	T_{max}	3868	3763	**3027**	3172	3063	3060	**3048**	**3042**
	Sum of total TTs	8412	7959	**4672**	5341	4828	4830	**4781**	**4738**
	CPUT	564.4	545.2	261.8	306.1	256	263.5	286.2	283.8
4	T_{max}	4808	4677	**3766**	3957	3818	3812	**3801**	**3787**
	Sum of total TTs	8310	7875	**4614**	5321	4791	4781	**4750**	**4703**
	CPUT	547.6	534.3	257.8	304.8	250	257.7	279.4	274.8
Freq. among best three		0	0	8	0	0	0	8	8
Gurobi gap in FSN		1%	1%	0%	0%	0%	0%	0%	0%

Table 7
Numerical results in solving the large-sized instances of $[OPMSSS]_{|R|}$ (i.e., $|R|$ varies from 6 to 10).

# of pickers	Criteria	ACDSS	WCDSS	ACMSSS	WCMSSS	RMSSS-0.25	RMSSS-0.5	RMSSS-0.75	RMSSS-1
10	T_{max}	4689	4557	**3363**	3527	**3367**	3388	3388	**3384**
	Sum of total TTs	27877	26648	**15093**	16674	**15142**	15335	15344	**15239**
	CPUT	1446.1	1417.1	490.4	754.8	639.1	614.9	522.1	502
9	T_{max}	5203	5060	**3728**	3909	**3736**	3750	3752	**3747**
	Sum of total TTs	27849	26599	**15051**	16642	**15145**	15251	15289	**15174**
	CPUT	1381.8	1349.7	452.3	727.3	587.9	560.7	464.2	457.7
8	T_{max}	5838	5690	**4190**	4393	**4195**	**4216**	4219	4216
	Sum of total TTs	27842	26602	**15049**	16630	**15137**	**15248**	15275	15249
	CPUT	1411.3	1377	464.4	739.5	597.7	567.1	475.1	464.4
7	T_{max}	6645	6470	**4784**	5015	**4794**	**4809**	4812	4813
	Sum of total TTs	27700	26449	**15020**	16626	**15129**	15234	15265	**15185**
	CPUT	1357.8	1329.9	443.4	721.3	575.8	548.7	455.1	449.6
6	T_{max}	7753	7543	**5576**	5848	**5587**	5610	5614	**5609**
	Sum of total TTs	27667	26463	**15029**	16643	**15123**	15226	15275	**15174**
	CPUT	1361.4	1332	445.6	721.9	577.7	547.7	451.9	451.4
Freq. among best three		0	0	10	0	10	3	0	7
Gurobi gap in FSN		2%	2%	0%	0%	0%	0%	0%	0%

5.2. Solving small, medium, and large-sized instances

In Tables 5, 6, and 7, we change the number of pickers and the SLA strategy to report two output types:
1- an algorithm-related criterion (CPUT): the average CPUT for solving a small/medium/large-sized instance of $[OPMSSS]_{|R|}$ using DOPBAR; and
2- two OP-related criteria: makespan and sum of the total TTs for all pickers (all in second).

Note that each value in these tables is an average obtained by solving 48 instances.

When the number of pickers is fixed, the OP-related values display the performance of different SLA strategies in response to the waves of orders in Tables 5, 6, and 7. For a given number of pickers, all of the MSSS-based SLAs outperform the DSS-based SLAs. In other words, applying a form of the MSSS for a warehouse results in a faster RT for a wave and shorter tours for the pickers. The three smallest values in terms of the OP-related criteria are bolded in these tables. In row "freq. among best three", for each SLA strategy, we report how frequent that SLA strategy has been among the best three SLA strategies. Although these tables do not suggest a particular SLA strategy for capturing the best OP process, the ACMSSS results in effective outcomes for OP-related criteria when our created order profiles apply. On the other hand, among the MSSS-based SLAs, the WCMSSS results in a lower performance in many cases.

Table 8
Reliability of DOPBAR's solutions in comparison with Gurobi's.

	T_{max} in Gurobi	T_{max} in DOPBAR
Mean	249	258
Variance	32234.3	35636.1
# of observations	60	60
p-value	0.3453	

We use Gurobi optimizer for solving BQPs (i.e., FSN formulated in Step 1). We set a 30-second time limit for the solver and report the average optimality gap left. These values are shown in row "Gurobi gap in FSN". Based on these values, FSNs can be solved effectively using a commercial optimizer within 30 s.

No values are reported regarding picker blocking when different SLA strategies apply as we have not focused on picker blocking in this paper. However, we aim to briefly elaborate on that based on our simulation experiments for implementing our OP plan on a warehouse with narrow aisles impeding picker movement (i.e., a picker cannot perform a pick in an aisle in which another picker is working). Our simulation experiments show that picker blocking can be reduced in MSSS-based warehouses. Hence, scattering SKUs to various storage locations may be helpful to reduce picker blocking. However, in order to certainly prove this statement, more methodological analyses and examinations are required.

Fig. 7 provides more details on the results shown in the last three tables. For small, medium, and large-sized instances, we set the number of pickers into 5, 7, and 10, respectively. Then, we present makespan values (in Figs. 7(a, b, c)) and sum of total TTs (in Figs. 7(d, e, f)) both in second for various order structures/profiles and SLA strategies. This figure reshows that MSSS-based warehouses outperform DSS-based warehouses. Moreover, by increasing the instance size, the importance of using the MSSS becomes more significant. In other words, as OP planning becomes more critical by increasing the warehouse size, the number of orders, and the number of SKUs requested, the MSSS plays a more crucial role to facilitate this process. Nevertheless, when the number of units that can be requested of the same SKU ($MaxI$) increases, the advantages of applying the MSSS instead of the DSS decrease. These kinds of orders are more similar to the orders that can be observed in B2B markets, and hence, the DSS becomes more prevalent for them.

5.3. Reliability analysis of the algorithm

As seen in the last subsection, DOPBAR is able to solve [OPMSSS] instances in a reasonable cut-off time. However, the lack of an algorithm for solving our MILP with its specific assumptions and minimizing the makespan is a reason that makes the evaluation of our solutions difficult. Therefore, we use a T-test to show that our solutions are statically close the solutions found by Gurobi optimizer. In this regard, we select a representative class of instances based on ACMSSS, $MaxSKU = 3$, and $MaxI = 3$, then solve 60 randomly generated instances to create our instance set for a comparison. We choose ACMSSS as this SLA strategy generally showed a good performance in many cases, and "$(3, 3)$" is the largest order profile of the test-sized instances. Note that all 60 instances are generated with the same SLA and order profile settings while the exact SLA and arrival orders in each instance may differ from one another. Table 8 reports the statistic values related to the T-test. As seen, a p-value of 0.34 is reported which statistically shows that there is no significant difference between DOPBAR's and Gurobi's solutions in a particular instance.

5.4. The effectiveness of solving FSN versus SPR and iterating the algorithm

One of crucial points in DOPBAR is the suitability of FSN in Step 1 in the design of the algorithm (remind discussions given in Section 3.2.1 and Appendix C). One may suggest that solving SPR problems can outperform FSN in the structure of DOPBAR and improve makespan values. In Table 9, we examine the effectiveness of both FSN and SPR for solving small-sized instances in five iterations ($MaxIter = 5$). Note that SLA strategies change by columns 3 to 10, and an average value is reported in the last column. In rows 2 to 9, we report values of makespan, sum of total TTs, and CPUT after 5 iterations of DOPBAR (in rows 2 to 7) using either FSNs or SPR problems; and in rows 8 and 9, we report makespan values obtained at the first iteration of DOPBAR. Note that the reported values of the makespan, sum of total TTs, and CPUT are based on the average of OP process with 3, 4, and 5 pickers.

In rows 10 and 11, we compare makespan/CPUT values based on using FSN and SPR. Table 9 displays that using FSN results in averagely 0.87% better makespan values comparing with solving SPR problems in Step 1. On the other hand, solving SPR problems is significantly more time consuming and requires averagely 13 times more CPUT. We also aim to examine positive influence of iterating DOPBAR in the last row of this table by comparing values in row 2 and row 8. We observe that makespan values can be averagely improved by 1.64% by 5 iterations. However, iterating DOPBAR for 5 times takes averagely 5 times longer than compiling DOPBAR for one iteration.

5.5. Trade-offs between makespan and workforce level

In general, we expect a shorter makespan when a larger number of pickers work together. Table 10 shows how much the makespan is averagely improved when we increase the number of pickers by one. Column two indicates the percentage of change in the workforce level based on adding one picker as shown in the first column. Columns three to ten refer to the average makespan improvements for each SLA strategy. Last two columns also present the average makespan improvement using the DSS and MSSS. In other words, the values in "avg. of DSS" are calculated by taking the average of the values reported in columns three and four; "avg. of MSSS" is the average of columns five to ten. These last two columns show that improvements in both MSSS-based SLAs and DSS-based SLAs are nearly the same. Our examination in Table 10 displays that when we increase the number of pickers by a certain percentage, the makespan will be improved with slightly a smaller percentage (i.e., the values in columns three to twelve are smaller than the values reported in column two). In simple terms, we cannot speed up an OP process as much as we hire more pickers.

We report makespan improvement values in Table 10 without considering aisle congestion that can be calculated by simulating created OP plans. Indeed, if we include picker blocking in our calculations, it is observed that larger improvements are obtainable under the MSSS because we face less picker blocking using this SLA strategy. Makespan improvements based on the MSSS can be 40% higher than improvements obtainable using the DSS.

6. Managerial and operational insights

In this paper, we propose a method for wave OP planning under the MSSS. As we examine the performance of various SLA strategies responding to different orders, a number of insightful suggestions for the managers and decision makers come into view. In general as seen in Tables 5, 6, and 7, for an online retailer with the suggested order profiles in this study, operating a warehouse under a form of the MSSS is faster (i.e., a shorter makespan), cheaper (i.e., a smaller total TT/distance), and more effective (i.e., the same workforce size gives a faster RT than the RT based on the DSS). Moreover, the mixture of across-aisle and the MSSS denoted by ACMSSS results in effective outcomes in a large subset of our instances (i.e., ACMSSS has been identified as one of the best three SLA strategies while solving instances associated with Tables 5, 6, and 7). Another point as a result of our discussion in Fig. 7 is about the benefits of using the MSSS; they are more significant when the variety of the requested SKUs is larger and the number of units requested of the same SKU is smaller.

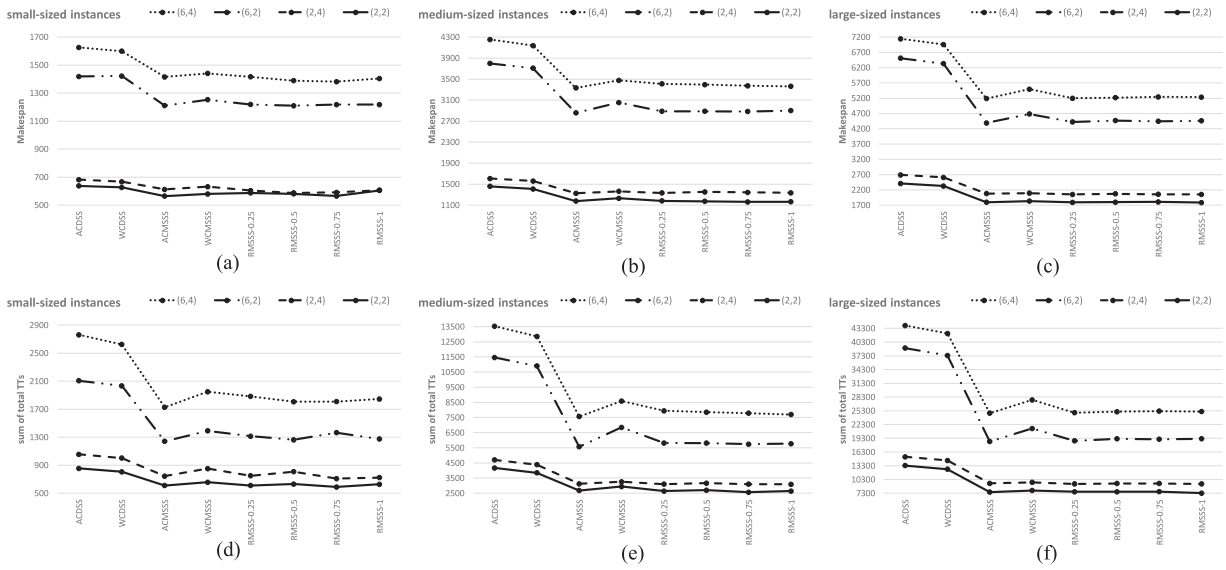


Fig. 7. T_{max} values and sum of total TTs in second based on various order behaviors and SLA strategies in small, medium, and large-sized instances with a given number of pickers.

Table 9
The impact of solving FSNs instead of SPR problems in Step 1 of DOPBAR and iterating the algorithm in small-sized instances.

Criteria & Methods		ACDSS	WCDSS	ACMSSS	WCMSSS	RMSSS-0.25	RMSSS-0.5	RMSSS-0.75	RMSSS-1	Avg.
T_{max}	FSN	1392.3	1372.5	1217.6	1240.9	1218.6	1195.6	1196.2	1213.5	1255.9
	SPR	1401.0	1383.4	1224.2	1252.1	1232.9	1208.7	1207.9	1223.5	1266.7
Sum of total TTs	FSN	1667.4	1586.7	1087.5	1178.7	1125.1	1100.1	1099.9	1093.7	1242.4
	SPR	1662.8	1598.0	1109.3	1183.0	1119.0	1079.6	1097.3	1115.9	1245.6
CPUT	FSN	153.0	145.1	83.9	88.6	88.7	91.1	94.3	93.1	104.7
	SPR	1032.1	1077.2	1285.2	1336.1	1385.5	1382.7	1363.6	1410.0	1284.1
T_{max} at 1st iter.	FSN	1428.9	1397.2	1230.6	1260.7	1236.5	1221.5	1210.4	1230.3	1277.0
	SPR	1431.1	1414.7	1248.6	1279.9	1264.3	1237.2	1241.1	1262.8	1297.5
FSN vs SPR	T_{max}	0.63%	0.80%	0.54%	0.90%	1.18%	1.10%	0.98%	0.83%	0.87%
	CPUT	674%	743%	1532%	1508%	1563%	1517%	1446%	1514%	1312%
T_{max} improved by 5 iter.		2.57%	1.77%	1.06%	1.57%	1.45%	2.12%	1.18%	1.37%	1.64%

Table 10
Improvements in T_{max} by changing the number of pickers.

Change in # of pickers	% change	ACDSS	WCDSS	ACMSSS	WCMSSS	RMSSS-0.25	RMSSS-0.5	RMSSS-0.75	RMSSS-1	Avg. of DSS	Avg. of MSSS
3 to 4	33.3%	30.53%	30.60%	30.53%	29.95%	30.63%	31.10%	30.73%	30.42%	30.56%	30.56%
4 to 5	25.0%	22.65%	22.08%	23.24%	22.30%	22.29%	21.48%	22.20%	21.38%	22.37%	22.15%
Avg.	29.2%	26.6%	26.3%	26.9%	26.1%	26.5%	26.3%	26.5%	25.9%	**26.46%**	**26.35%**
4 to 5	25.0%	24.31%	24.31%	24.40%	24.75%	24.64%	24.61%	24.69%	24.48%	24.31%	24.59%
5 to 6	20.0%	19.58%	19.37%	19.62%	19.50%	19.72%	19.67%	19.60%	19.50%	19.48%	19.60%
6 to 7	16.7%	16.35%	16.58%	16.43%	16.34%	16.15%	16.08%	16.34%	16.20%	16.47%	16.26%
Avg.	20.6%	20.1%	20.1%	20.1%	20.2%	20.2%	20.1%	20.2%	20.1%	**20.08%**	**20.15%**
6 to 7	16.7%	16.66%	16.58%	16.56%	16.62%	16.53%	16.65%	16.67%	16.53%	16.62%	16.59%
7 to 8	14.3%	13.82%	13.70%	14.17%	14.14%	14.28%	14.08%	14.04%	14.16%	13.76%	14.14%
8 to 9	12.5%	12.22%	12.45%	12.39%	12.39%	12.28%	12.41%	12.44%	12.51%	12.33%	12.40%
9 to 10	11.1%	10.97%	11.04%	10.86%	10.84%	10.95%	10.68%	10.76%	10.74%	11.00%	10.80%
Avg.	13.6%	13.4%	13.4%	13.5%	13.5%	13.5%	13.5%	13.5%	13.5%	**13.4%**	**13.5%**

In general, increasing the workforce level in the OP systems does not improve the makespan as much as the percentage increment in the workforce level. Without including aisle congestion, we can obtain the same improvements by adding to the number of pickers in both MSSS-based and DSS-based warehouses. However, our simulation experiments show that the number of picker blocking reduces under the MSSS (discussion given in Table 10).

The design of DOPBAR and its steps are established on basic computations and solving mathematical models. Either this structure or a customized version of our method can be straightforwardly implemented for wave OP planning under the MSSS. DOPBAR solves problem instances in a reasonable cut-off time and finds effective solutions (e.g., a 30-minute cut-off time is recommended by Ardjmand et al. (2018) for a wave OP process).

7. Conclusions and future research directions

In this paper, we integrate three wave OP sub-planning problems, batching orders, batch assignment, and routing pickers, under the MSSS

while multiple pickers work simultaneously. We propose a bi-objective MILP formulation called [OPMSSS] that finds the optimal batching for a wave of orders to minimize the makespan and the workforce level. We design a method, DOPBAR, for finding an effective solution for an instance of [OPMSSS] when the workforce level is fixed.

Our examination shows that when the structure of orders is as the orders usually observed in the e-commerce markets, MSSS-based SLAs are beneficial for the warehouses. Those benefits include shorter TTs for the pickers, shorter makespan, and achieving higher improvements in OP (i.e., a faster and less costly OP process) by increasing the number of pickers. Hence, applying a form of the MSSS is recommended for handling warehouse operations.

The MSSS has being applied for many warehouses, particularly in online retailing. In terms of optimization, this strategy adds computational difficulties to the OP planning problems, and hence, there is a narrow literature on that. Despite few existing papers and our work, there are research directions related to the MSSS that must be scrutinized in the future; we list a few of them as follows:

- Collecting several waves with different deadlines simultaneously.
- How much effective a MSSS-based warehouse operates if some orders include tens of units of the same SKU (i.e., an order structure/profile that can be observed in B2B markets)? How to develop a SLA that efficiently handles arriving orders with a heterogenous behavior?
- Evaluating the level of picker blocking more accurately and including that in a mathematical model for OP under the MSSS.
- Examining the effectiveness of the MSSS for a warehouse with multiple blocks.
- Evaluating the efficiency of our algorithm for different warehouse layouts like the existence of multiple blocks.
- Developing new methods and improving DOPBAR for OP planning under the MSSS.

CRediT authorship contribution statement

Seyyed Amir Babak Rasmi: Conceptualization, Methodology (Mathematical Modeling, Heuristic, and implementation), Validation, Formal analysis, Investigation, Visualization, Writing (original draft, review & editing). **Yuan Wang:** Conceptualization, Methodology, Writing (review & editing), Formal analysis. **Hadi Charkhgard:** Methodology, Writing (review & editing), Formal analysis.

Acknowledgments

The authors have been supported by Singapore A*STAR IAF-PP fund (Grant No. A1895a0033) under the project "Digital Twin for Next Generation Warehouse".

Appendix A. An illustrative example on batching and routing for two pickers under the MSSS

Fig. A.1 elaborates on the importance of joint decision-making for order batching and picker routing. Assume that the SLA and the number of units available are the same as Fig. 1(b). Let "$*$ (#)" denote "# number of unit(s) requested from SKU $*$", and four orders have been received as seen in Fig. A.1(a). Assume that two pickers are available, and we aim to let each of them retrieve one batch of orders. Therefore, in order to comply batching, we partition the set of orders to two non-empty subsets (i.e., Batch 1 and Batch 2). Let order batching be done in two ways; Order Batching Plans 1 and 2 as shown in Figs. A.1(b) and A.1(c), respectively. Using white numbered circles and triangles, we denote the picking nodes in which there is at least a pick related to Batch 1 and Batch 2, respectively. Hence, a feasible routing solution for coordinating Pickers 1 and 2 using Order Batching Plan 1 (Fig. A.1(b)) is to visit picking nodes ①-… -④and △1-… -△4, respectively. Routing plan changes when Order Batching Plan 2 applies (Fig. A.1(c)). Assume that $g = u = 1$ and $m = 3$ time units in Fig. 1(c). Given that, if we follow Order Batching Plan 1, Batch 1's total TT is 18 and Batch 2's total TT is 26. However, applying Order Batching Plan 2 results in 22 and 28 time units for the total TTs of Batches 1 and 2, respectively.

Note that for the illustrative example discussed in Fig. A.1, seven different order batching plans exist to create two batches of orders (i.e., partitioning set $\{1, \dots, 4\}$ to two non-empty subsets). Each order bathing plan results in a different total TT and makespan.

Appendix B. Additional Constraints for [OPMSSS]

There are various symmetry breaking constraints in the literature of OP and vehicle routing problems (e.g., see Van Gils et al. (2019a) and Valle et al. (2017)). We mention the following constraints that are beneficial due to the nature of our problem.

$$W_{b-1} \geq W_b, \ \forall b \in B \backslash \{1\}, \tag{B.1}$$

$$T^{b-1} \geq T^b, \ \forall b \in B \backslash \{1\}, \tag{B.2}$$

$$A_{r-1} \geq A_r, \ r \in R \backslash \{1\}, \tag{B.3}$$

$$\bar{T}^{r-1} \geq \bar{T}^r, \ r \in R \backslash \{1\}, \tag{B.4}$$

where batch b (picker r) can become active if batch $b-1$ (picker $r-1$) is active (constraints (B.1) and (B.3)). Moreover, constraints (B.2) and (B.4) show that the total RTs of batches and pickers are non-increasing with respect to their indices, respectively. In order to clarify the advantage of these constraints other than tightening the feasible region, let a certain number of pickers work in a warehouse to retrieve a wave of orders. Then, for the next wave, we can assign the longest retrieval activity to the picker who has completed picking activities with the shortest RT in the previous wave OP process, and vice versa.

We linearize constraints (17) by replacing them with the following constraints given in (B.5) as U_b^r's are binary and T^b's are continuous decision variables:

$$\begin{aligned}
&\sum_{b \in B} \hat{T}^{br} \leq \bar{T}^r, \ \forall r \in R, \\
&\hat{T}^{br} \leq T^b, \ \forall b \in B, r \in R, \\
&\hat{T}^{br} \leq \hat{M} U_b^r, \ \forall b \in B, r \in R, \\
&\hat{T}^{br} \geq T^b - \hat{M}(1 - U_b^r), \ \forall b \in B, r \in R, \\
&\hat{T}^{br} \geq 0, \ \forall b \in B, r \in R.
\end{aligned} \tag{B.5}$$

We aim to propose another set of inequalities that strengthen constraints (6). The following constraint for a certain $r \in R$ expresses that the maximum number of batches which can be assigned to picker r is smaller than the number of active batches (i.e., $\sum_{b \in B} W_b$). On the other hand, if there exist other active pickers except r, each of them handles at least one batch. Therefore, we deduct $\sum_{r' \in R \backslash \{r\}} A_{r'}$ from the right-hand-side of the constraint.

$$\sum_{b \in B} U_b^r \leq \sum_{b \in B} W_b - \sum_{r' \in R \backslash \{r\}} A_{r'}, \ \forall r \in R. \tag{B.6}$$

Appendix C. SPR Formulation Based on Weidinger (2018)

Weidinger (2018) proposes a mathematical model to find the minimum TT for a picker retrieving one or multiple orders in a MSSS-based warehouse. The model with a detailed description is available in his paper. In this section, we rewrite that model using our notation and leave the model description to the readers. Moreover, we add picking times at the storage locations to the objective function proposed by Weidinger (2018) as that only includes the total TT.

Assume that we aim to model SPR for collecting order $\hat{o}$ in minimum total RT, and hence, $I := S_{\hat{o}}$ (i.e., we reduce the set of SKUs). Let v_0^0 denote the depot for both leaving and returning picker (i.e., we merge $\{0^s\}$ and $\{0^e\}$ into $\{0\}$ and set $I_0 = \{0\}$). In order to establish our SPR($\hat{o}$, LCI) model derived from Weidinger (2018), we set $b := 1$ and

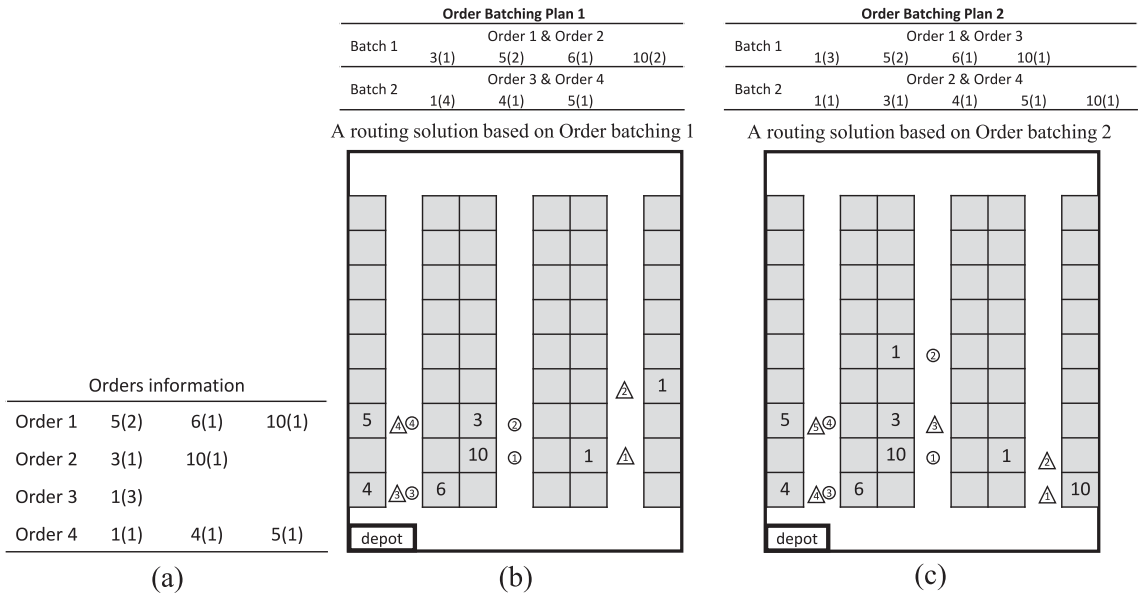


Fig. A.1. An example to illustrate how order batching influences routing in an OP process (order information is shown in (a), the tours corresponding to Order Batching Plans 1 and 2 are shown in (b) and (c), respectively).

update the definition of $X^b_{v_i^k v_j^p}$, $\forall i \in I \cup \{0\}$, $k \in I_i$, $j \in I \cup \{0\}$, $p \in I_j$; it takes the value of 1, if storage location v_j^p is visited immediately after storage location v_i^k; otherwise, 0. $X^b_{v_i^k v_i^k}$ also becomes 1 when storage location v_i^k is not visited. Otherwise, it takes the values of 0. Based on this definition, in total, we have n nodes including the depot, and then, $n := 1 + \sum_{i \in I} |I_i|$. We present SPR($\hat{o}$, LCI) as follows:

SPR($\hat{o}$, LCI) :

$$\min \sum_{i \in I \cup \{0\}} \sum_{j \in I \cup \{0\}} \sum_{k \in I_i} \sum_{p \in I_j} \tau_{v_i^k v_j^p} X^b_{v_i^k v_j^p} + \tau_{pick} \sum_{i \in I} \sum_{k \in I_i} (1 - X^b_{v_i^k v_i^k}) \tag{C.1}$$

$$\text{s.t.} \sum_{i \in I \cup \{0\}} \sum_{k \in I_i} X^b_{v_i^k v_j^p} = 1, \ \forall j \in I \cup \{0\}, p \in I_j, \tag{C.2}$$

$$\sum_{j \in I \cup \{0\}} \sum_{p \in I_j} X^b_{v_i^k v_j^p} = 1, \ \forall i \in I \cup \{0\}, k \in I_i, \tag{C.3}$$

$$\sum_{k \in I_i} (\hat{q}_i^k X^b_{v_i^k v_i^k}) \le \sum_{k \in I_i} \hat{q}_i^k - n^i_{\hat{o}}, \ \forall i \in I, \tag{C.4}$$

$$\sum_{p \in I_i \setminus \{k\}} (\hat{q}_i^p X^b_{v_i^p v_i^p}) + X^b_{v_i^k v_i^k} \sum_{p \in I_i} \hat{q}_i^p \ge \sum_{p \in I_i} \hat{q}_i^p - n^i_{\hat{o}} - \hat{q}_i^k + 1, \ \forall i \in I, k \in I_i, \tag{C.5}$$

$$V_{v_i^k} - V_{v_j^p} + n X^b_{v_i^k v_j^p} - n(X^b_{v_i^k v_i^k} + X^b_{v_j^p v_j^p}) \le n - 1, \ \forall i \in I, k \in I_i, j \in I, p \in I_j, \tag{C.6}$$

$$V_{v_i^k} \in \mathbb{Z}^+ \cup \{0\}, \ \forall i \in I, k \in I_i, \tag{C.7}$$

$$X^b_{v_i^k v_j^p} \in \{0,1\}, \ \forall i \in I \cup \{0\}, k \in I_i, \ j \in I \cup \{0\}, p \in I_j. \tag{C.8}$$

When we use SPR($\hat{o}$, LCI) instead of FSN($\hat{o}$, LCI) in line 6 of Procedure 1, according to the new definition of $X^b_{v_i^k v_i^k}$'s, the value of $\lambda^{\hat{o}}_{ik}$ is set into one minus the optimal solution of $X^b_{v_i^k v_i^k}$ (i.e., $\lambda^{\hat{o}}_{ik} := (1 - X^b_{v_i^k v_i^k})$).

Appendix D. How to generate a SLA instance based on the RMSSS-β, ACMSSS, and WCMSSS

In Procedure D.1, we explain how to generate a RMSSS-β-based SLA instance. Let $\bar{q}$ be the maximum number of items that can be placed in a bin. We number all storage locations starting from 1 and initialize the set of vacant storage locations, VSL (line 1). In order to guarantee that our inventory includes some units of all SKUs, we randomly assign a vacant storage location to each SKU within lines 2–4. Note that the set of vacant bins is updated in line 5. The number of storage locations is usually greater than the number of SKUs. Hence, in each iteration of the loop given in line 6, we need to assign a SKU to a bin which is still vacant. In line 7, we select a SKU randomly (e.g., i) according to the probability distribution in the class-based categorization (i.e., 80%, 15%, and 5% for A, B, and C-class products). Then, we find the storage locations which are occupied by SKU i (line 8). In line 9, we create the corresponding TT matrix as shown in Fig. 6(b). In line 10, we select a vacant bin according to the weighted-sum TTs. Then, we assign SKU i to that bin and update VSL in lines 11–12. We follow the same approach until all bins are occupied.

Procedure D.1 Generate a SLA instance based on the RMSSS-β.
Input: (β), output: (a SLA instance).

1: Initialize VSL as $\{1, 2, \ldots,$ total number of bins$\}$;
2: **for** each $i \in I$, **do**
3: select an element from VSL and denote it by v;
4: assign a random number* of units of SKU i to v;
5: $VSL := VSL - \{v\}$;
6: **while** $VSL \neq \emptyset$, **do**
7: select a SKU such as $i \in I$ randomly according to the probability distribution in the class-based categorization.
8: Let $ASL(i)$ be the set of the storage locations to which SKU i has been assigned.
9: Create the TT matrix from the bins in $ASL(i)$ to the bins in VSL (see Fig. 6(b)).
10: Calculate the weighted-sum TTs and select a vacant bin such as $v \in VSL$ based on the value of β (see Fig. 6(c)).
11: Assign a random number* of units of SKU i to v.
12: $VSL := VSL - \{v\}$;
13: **return** a SLA instance based on the number of units of a SKU that has been assigned to each storage location.

* A uniformly distributed random number from the set $\{1, \ldots, \bar{q}\}$.

In this paper, for generating SLA instances based on the WCMSSS/ACMSSS, we set a region for the products of each class. In Fig. D.1(a), the light gray bins show the storage locations allocated to the A-class products when the WCMSSS applies (see Fig. D.1(b) when the ACMSSS applies). The storage locations for the B-class and C-class products are colored in dark gray and black, respectively. In

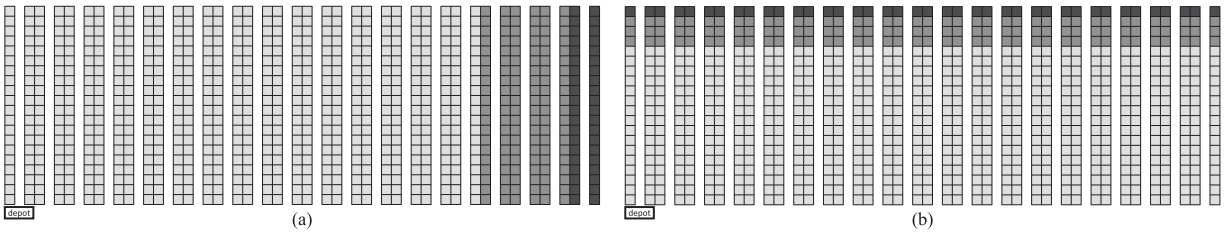


Fig. D.1. An illustration on the bins allocated to the A-class (light gray), B-class (dark gray), and C-class (black) products under the (a) WCMSSS and (b) ACMSSS.

this regard, assume that we are assigning bins to X-class products (i.e., X ∈ {A,B,C}). Then, we randomly select an X-class SKU, randomly select a vacant bin that is located in the region allocated to the X-class products, and assign a number of units to that bin. We do not consider the scattering degree in generating the SLA instances under the WCMSSS and ACMSSS. One may generate these instances following the approach used in Procedure D.1.

Appendix E. [OPDSS]: [OPMSSS] When the storage locations for picking are preselected

In this section, we present a bi-objective MILP that is originated from [OPMSSS] and optimizes OP under the DSS. In order to keep notation in consistent with our model in [OPMSSS], we assume that each storage location such as v_i^k denotes a different SKU (i.e., the number of SKUs is equal to the number of storage locations). Based on this definition, we revise set I as the set of all SKUs, and hence, $I_i := \{0\}$ for all $i \in I$ as only one storage location for each SKU exists. We also need to revise n_o^i's accordingly. For example, assume that it has been decided to visit bin v_i^k for order o to collect q number of units. Then, n_o^i will take the value of q where i corresponds to the new SKU defined for bin v_i^k. We formulate [OPDSS] as follows:

[**OPDSS**] :

$$\min \ (1) \tag{E.1}$$

$$\min \ (2) \tag{E.2}$$

$$\text{s.t. } (3), (4), (5), (6), (7), (8), (9), (10), \tag{E.3}$$
$$(15), (16), (17), (18), (19), (20), (22),$$

$$\Big(\sum_{o\in O} n_o^i\Big)\Big(\sum_{j\in I\cup\{0^e\}}\ \sum_{p\in I_j\setminus\{k:j=i\}} X^b_{v_i^k v_j^p}\Big) \geq \sum_{o\in O} n_o^i Y_b^o, \tag{E.4}$$
$$\forall b \in B, i \in I, k \in I_i,$$

$$\sum_{j\in I\cup\{0^e\}}\ \sum_{p\in I_j\setminus\{k:j=i\}} X^b_{v_i^k v_j^p} \leq \sum_{o\in O} n_o^i Y_b^o, \ \forall b \in B, i \in I, k \in I_i. \tag{E.5}$$

As seen, [OPDSS] is only different from [OPMSSS] in variables Q_{ib}^{ko}'s. Because we know the storage locations that must be visited, and hence, the number of units that need to be picked from each storage location is not a question. Therefore, we exclude constraints (11), (12), and (21); we also add constraints (E.4) and (E.5) instead of (13) and (14), respectively.

Appendix F. Primal bound improvement with respect to time in solving test-sized instances by Gurobi

As mentioned at the end of Section 5.1, we have observed that global primal bound improves rapidly when optimization begins. However, its improvement speed significantly decreases by time. In Fig. F.1, we depict a box plot to compare primal bound improvements in the first 10 min (i.e., from the time a feasible solution has been found until the end of the 10th minute) with the next 50-minute time slot after that. Although the median of improvements in the first 10 min is 73.5%, this value reduces to 0.8% after 10 min up to one hour. This evidence supports the fact that the solutions obtained by Gurobi optimizer are (nearly) optimal after one hour of solving test-sized instances.

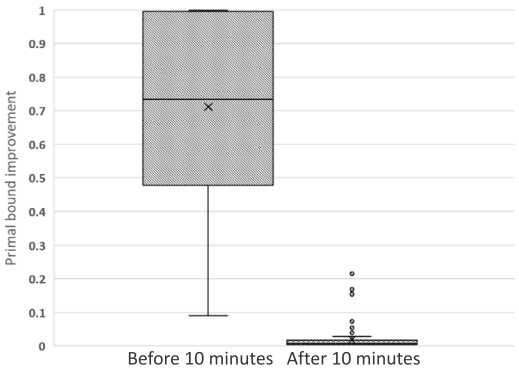


Fig. F.1. Primal bound improvement in the first 10-minute time slot and after the 10th minute to 1 h of solving test-sized instances.

References

Adams, W.P., Sherali, H.D., 1986. A tight linearization and an algorithm for zero-one quadratic programming problems. Manage. Sci. 32 (10), 1274–1290.

Ardjmand, E., Shakeri, H., Singh, M., Bajgiran, O.S., 2018. Minimizing order picking makespan with multiple pickers in a wave picking warehouse. Int. J. Prod. Econ. 206, 169–183.

Azadnia, A.H., Taheri, S., Ghadimi, P., Mat Saman, M.Z., Wong, K.Y., 2013. Order batching in warehouses by minimizing total tardiness: a hybrid approach of weighted association rule mining and genetic algorithms. Sci. World J. 2013.

Billionnet, A., Elloumi, S., Plateau, M.-C., 2005. Convex quadratic programming for exact solution of 0-1 quadratic programs. RAIRO-Oper. Res..

Boysen, N., de Koster, R., Weidinger, F., 2019. Warehousing in the e-commerce era: A survey. European J. Oper. Res. 277 (2), 396–411.

Briant, O., Cambazard, H., Cattaruzza, D., Catusse, N., Ladier, A.-L., Ogier, M., 2020. An efficient and general approach for the joint order batching and picker routing problem. European J. Oper. Res. 285 (2), 497–512.

Cano, J.A., Correa-Espinal, A.A., Gómez-Montoya, R.A., 2019. Mathematical programming modeling for joint order batching, sequencing and picker routing problems in manual order picking systems. J. King Saud Univ., Eng. Sci..

Caprara, A., 2008. Constrained 0–1 quadratic programming: Basic approaches and extensions. European J. Oper. Res. 187 (3), 1494–1503.

Chen, L., Langevin, A., Riopel, D., 2010. The storage location assignment and interleaving problem in an automated storage/retrieval system with shared storage. Int. J. Prod. Res. 48 (4), 991–1011.

Chen, F., Wei, Y., Wang, H., 2018. A heuristic based batching and assigning method for online customer orders. Flex. Serv. Manuf. J. 30 (4), 640–685.

Chen, M.-C., Wu, H.-P., 2005. An association-based clustering approach to order batching considering customer demand patterns. Omega 33 (4), 333–343.

Cheng, C.-Y., Chen, Y.-Y., Chen, T.-L., Yoo, J.J.-W., 2015. Using a hybrid approach based on the particle swarm optimization and ant colony optimization to solve a joint order batching and picker routing problem. Int. J. Prod. Econ. 170, 805–814.

Cortés, P., Gómez-Montoya, R.A., Muñuzuri, J., Correa-Espinal, A., 2017. A tabu search approach to solving the picking routing problem for large-and medium-size distribution centres considering the availability of inventory and K heterogeneous material handling equipment. Appl. Soft Comput. 53, 61–73.

Daniels, R.L., Rummel, J.L., Schantz, R., 1998. A model for warehouse order picking. European J. Oper. Res. 105 (1), 1–17.

De Koster, R., Le-Duc, T., Roodbergen, K.J., 2007. Design and control of warehouse order picking: A literature review. European J. Oper. Res. 182 (2), 481–501.
Dijkstra, A.S., Roodbergen, K.J., 2017. Exact route-length formulas and a storage location assignment heuristic for picker-to-parts warehouses. Transp. Res. E 102, 38–59.
Ehrgott, M., 2000. Multicriteria Optimization. In: Lecture Notes in Economics and Mathematical Systems, vol. 491, Springer, Berlin.
Gil-Borrás, S., Pardo, E.G., Alonso-Ayuso, A., Duarte, A., 2020. GRASP with variable neighborhood descent for the online order batching problem. J. Global Optim. 78, 295–325.
Glover, F., Woolsey, E., 1974. Converting the 0-1 polynomial programming problem to a 0-1 linear program. Oper. Res. 22 (1), 180–182.
Gu, S., Chen, X., 2020. The basic algorithm for the constrained zero-one quadratic programming problem with k-diagonal matrix and its application in the power system. Mathematics 8 (1), 138.
Gu, J., Goetschalckx, M., McGinnis, L.F., 2007. Research on warehouse operation: A comprehensive review. European J. Oper. Res. 177 (1), 1–21.
Haouassi, M., Kergosien, Y., Mendoza, J.E., Rousseau, L.-M., 2021. The integrated orderline batching, batch scheduling, and picker routing problem with multiple pickers: the benefits of splitting customer orders. Flex. Serv. Manuf. J. 1–32.
Henn, S., 2015. Order batching and sequencing for the minimization of the total tardiness in picker-to-part warehouses. Flex. Serv. Manuf. J. 27 (1), 86–114.
Hong, S., Johnson, A.L., Peters, B.A., 2012a. Batch picking in narrow-aisle order picking systems with consideration for picker blocking. European J. Oper. Res. 221 (3), 557–570.
Hong, S., Johnson, A.L., Peters, B.A., 2012b. Large-scale order batching in parallel-aisle picking systems. IIE Trans. 44 (2), 88–106.
Jiang, W., Liu, J., Dong, Y., Wang, L., 2020. Assignment of duplicate storage locations in distribution centres to minimise walking distance in order picking. Int. J. Prod. Res. 1–15.
Kulak, O., Sahin, Y., Taner, M.E., 2012. Joint order batching and picker routing in single and multiple-cross-aisle warehouses using cluster-based Tabu search algorithms. Flex. Serv. Manuf. J. 24 (1), 52–80.
Larco, J.A., De Koster, R., Roodbergen, K.J., Dul, J., 2017. Managing warehouse efficiency and worker discomfort through enhanced storage assignment decisions. Int. J. Prod. Res. 55 (21), 6407–6422.
Lenstra, J.K., Shmoys, D.B., Tardos, E., 1990. Approximation algorithms for scheduling unrelated parallel machines. Math. Program. 46 (1), 259–271.
Li, J., Huang, R., Dai, J.B., 2017. Joint optimisation of order batching and picker routing in the online retailer's warehouse in China. Int. J. Prod. Res. 55 (2), 447–461.
Lin, S., Kernighan, B.W., 1973. An effective heuristic algorithm for the traveling-salesman problem. Oper. Res. 21 (2), 498–516.
Lin, Y.-K., Pfund, M.E., Fowler, J.W., 2011. Heuristics for minimizing regular performance measures in unrelated parallel machine scheduling problems. Comput. Oper. Res. 38 (6), 901–916.
Lloyd, S., 1982. Least squares quantization in PCM. IEEE Trans. Inform. Theory 28 (2), 129–137.
Lu, W., McFarlane, D., Giannikas, V., Zhang, Q., 2016. An algorithm for dynamic order-picking in warehouse operations. European J. Oper. Res. 248 (1), 107–122.
Matusiak, M., de Koster, R., Kroon, L., Saarinen, J., 2014. A fast simulated annealing method for batching precedence-constrained customer orders in a warehouse. European J. Oper. Res. 236 (3), 968–977.
Menéndez, B., Bustillo, M., Pardo, E.G., Duarte, A., 2017. General variable neighborhood search for the order batching and sequencing problem. European J. Oper. Res. 263 (1), 82–93.
Miguel, F., Frutos, M., Tohmé, F., Rossit, D., 2019. A memetic algorithm for the integral OBP/OPP problem in a logistics distribution center. Uncertain Suppl. Chain Manage. 7 (2), 203–214.
Pan, J.C.-H., Wu, M.-H., 2012. Throughput analysis for order picking system with multiple pickers and aisle congestion considerations. Comput. Oper. Res. 39 (7), 1661–1672.
Pang, K.-W., Chan, H.-L., 2017. Data mining-based algorithm for storage location assignment in a randomised warehouse. Int. J. Prod. Res. 55 (14), 4035–4052.
Parikh, P.J., Meller, R.D., 2008. Selecting between batch and zone order picking strategies in a distribution center. Transp. Res. E 44 (5), 696–719.
Petersen, C.G., Aase, G., 2004. A comparison of picking, storage, and routing policies in manual order picking. Int. J. Prod. Econ. 92 (1), 11–19.
Petersen, C.G., Aase, G.R., et al., 2017. Improving order picking efficiency with the use of cross aisles and storage policies. Open J. Bus. Manage. 5 (01), 95.
Pfund, M., Fowler, J.W., Gupta, J.N., 2004. A survey of algorithms for single and multi-objective unrelated parallel-machine deterministic scheduling problems. J. Chin. Inst. Ind. Eng. 21 (3), 230–241.
Quader, S., Castillo-Villar, K.K., 2018. Design of an enhanced multi-aisle order-picking system considering storage assignments and routing heuristics. Robot. Comput.-Integr. Manuf. 50, 13–29.
Ratliff, H.D., Rosenthal, A.S., 1983. Order-picking in a rectangular warehouse: a solvable case of the traveling salesman problem. Oper. Res. 31 (3), 507–521.
Reyes, J., Solano-Charris, E., Montoya-Torres, J., 2019. The storage location assignment problem: A literature review. Int. J. Ind. Eng. Comput. 10 (2), 199–224.
Roodbergen, K.J., 2012. Storage assignment for order picking in multiple-block warehouses. In: Manzini, R. (Ed.), Warehousing in the Global Supply Chain: Advanced Models, Tools and Applications for Storage Systems. Springer London, London, pp. 139–155.
Roodbergen, K.J., Koster, R., 2001. Routing methods for warehouses with multiple cross aisles. Int. J. Prod. Res. 39 (9), 1865–1883.
Scholz, A., Schubert, D., Wäscher, G., 2017. Order picking with multiple pickers and due dates–Simultaneous solution of Order Batching, Batch Assignment and Sequencing, and Picker Routing Problems. European J. Oper. Res. 263 (2), 461–478.
Silva, A., Coelho, L.C., Darvish, M., Renaud, J., 2020. Integrating storage location and order picking problems in warehouse planning. Transp. Res. E 140, 102003.
Theys, C., Bräysy, O., Dullaert, W., Raa, B., 2010. Using a TSP heuristic for routing order pickers in warehouses. European J. Oper. Res. 200 (3), 755–763.
Unlu, Y., Mason, S.J., 2010. Evaluation of mixed integer programming formulations for non-preemptive parallel machine scheduling problems. Comput. Ind. Eng. 58 (4), 785–800.
Valle, C.A., Beasley, J.E., da Cunha, A.S., 2017. Optimally solving the joint order batching and picker routing problem. European J. Oper. Res. 262 (3), 817–834.
Van Gils, T., Caris, A., Ramaekers, K., Braekers, K., 2019a. Formulating and solving the integrated batching, routing, and picker scheduling problem in a real-life spare parts warehouse. European J. Oper. Res. 277 (3), 814–830.
Van Gils, T., Caris, A., Ramaekers, K., Braekers, K., de Koster, R.B., 2019b. Designing efficient order picking systems: The effect of real-life features on the relationship among planning problems. Transp. Res. E 125, 47–73.
Van Gils, T., Ramaekers, K., Braekers, K., Depaire, B., Caris, A., 2018a. Increasing order picking efficiency by integrating storage, batching, zone picking, and routing policy decisions. Int. J. Prod. Econ. 197, 243–261.
Van Gils, T., Ramaekers, K., Caris, A., de Koster, R.B., 2018b. Designing efficient order picking systems by combining planning problems: State-of-the-art classification and review. European J. Oper. Res. 267 (1), 1–15.
Weidinger, F., 2018. Picker routing in rectangular mixed shelves warehouses. Comput. Oper. Res. 95, 139–150.
Weidinger, F., Boysen, N., 2018. Scattered storage: how to distribute stock keeping units all around a mixed-shelves warehouse. Transp. Sci. 52 (6), 1412–1427.
Weidinger, F., Boysen, N., Schneider, M., 2019. Picker routing in the mixed-shelves warehouses of e-commerce retailers. European J. Oper. Res. 274 (2), 501–515.
Yang, P., Zhao, Z., Guo, H., 2020. Order batch picking optimization under different storage scenarios for e-commerce warehouses. Transp. Res. E 136, 101897.
Yener, F., Yazgan, H.R., 2019. Optimal warehouse design: Literature review and case study application. Comput. Ind. Eng. 129, 1–13.
Yousefi Nejad Attari, M., Ebadi Torkayesh, A., Malmir, B., Neyshabouri Jami, E., 2020. Robust possibilistic programming for joint order batching and picker routing problem in warehouse management. Int. J. Prod. Res. 1–19.
Zhang, J., Wang, X., Chan, F.T., Ruan, J., 2017. On-line order batching and sequencing problem with multiple pickers: A hybrid rule-based algorithm. Appl. Math. Model. 45, 271–284.
Žulj, I., Kramer, S., Schneider, M., 2018. A hybrid of adaptive large neighborhood search and Tabu search for the order-batching problem. European J. Oper. Res. 264 (2), 653–664.